



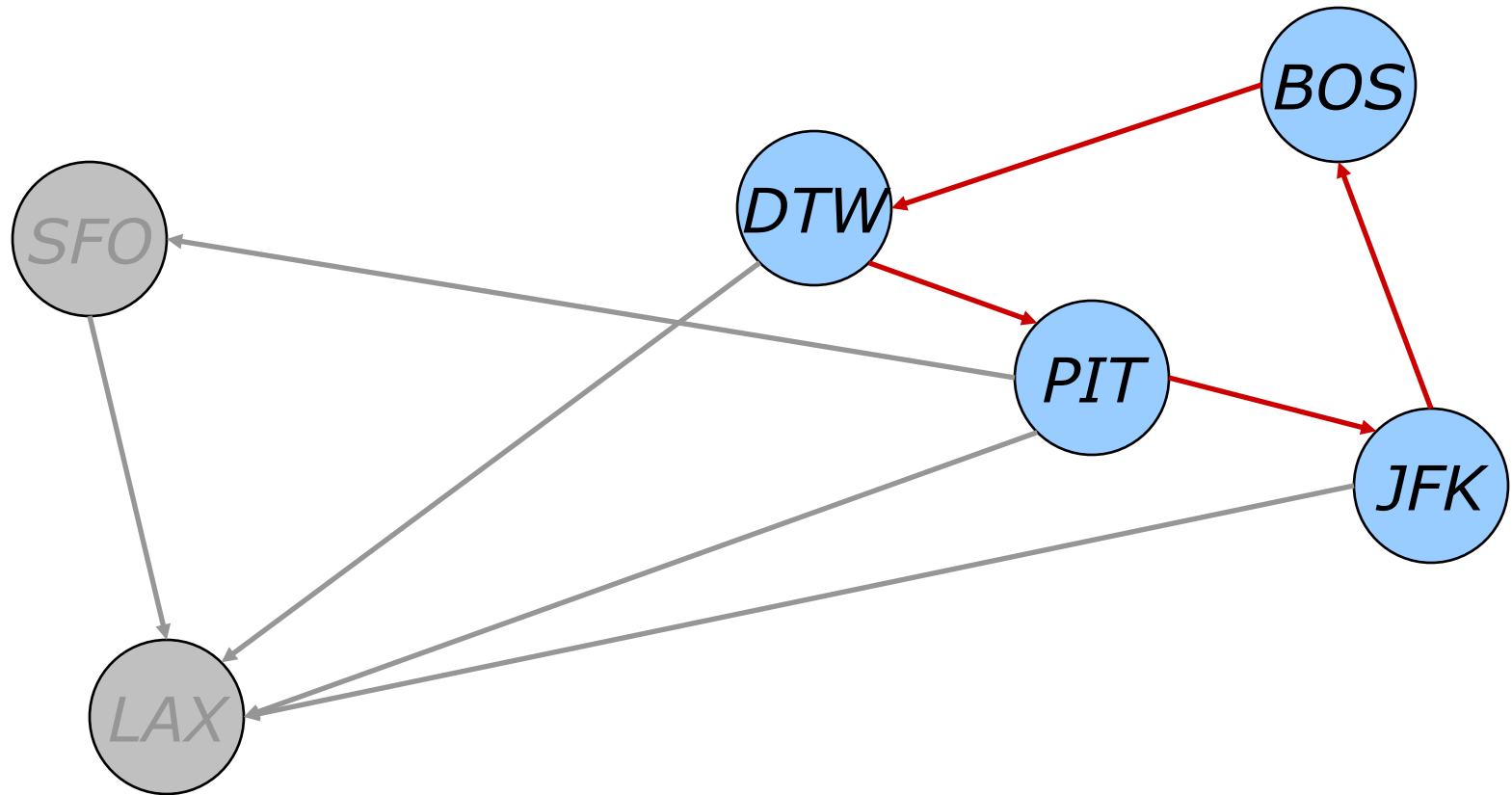
Algorithms

Graphs

Paths and cycles

- A *path* is a sequence of nodes $v_1, v_2, \dots, v_N$ such that $(v_i, v_{i+1}) \in E$ for $0 < i < N$
 - The *length* of the path is $N-1$.
 - *Simple path*: all v_i are distinct, $0 < i < N$
- A *cycle* is a path such that $v_1 = v_N$
 - An *acyclic* graph has no cycles

Cycles



More useful definitions

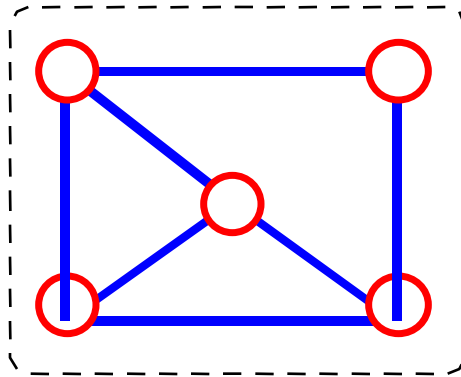
- The **degree** of a vertex is the number of edges incident to that vertex
- In a directed graph:
 - The *indegree* of a node v is the number of distinct edges $(w,v) \in E$.
 - The *outdegree* of a node v is the number of distinct edges $(v,w) \in E$.
- A node with indegree 0 is a *root*.

Trees are graphs

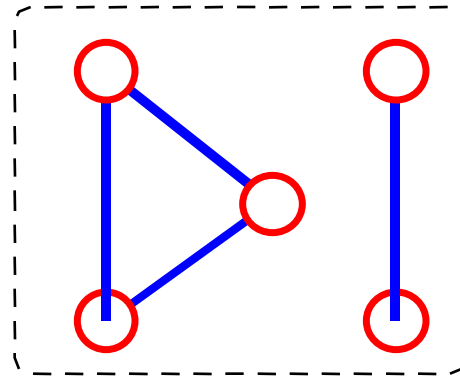
- A *dag* is a directed acyclic graph.
- A *tree* is a connected acyclic undirected graph.
- A *forest* is an acyclic undirected graph (not necessarily connected), i.e., each connected component is a tree.

Even More Terminology

- **connected graph**: any two vertices are connected by some path

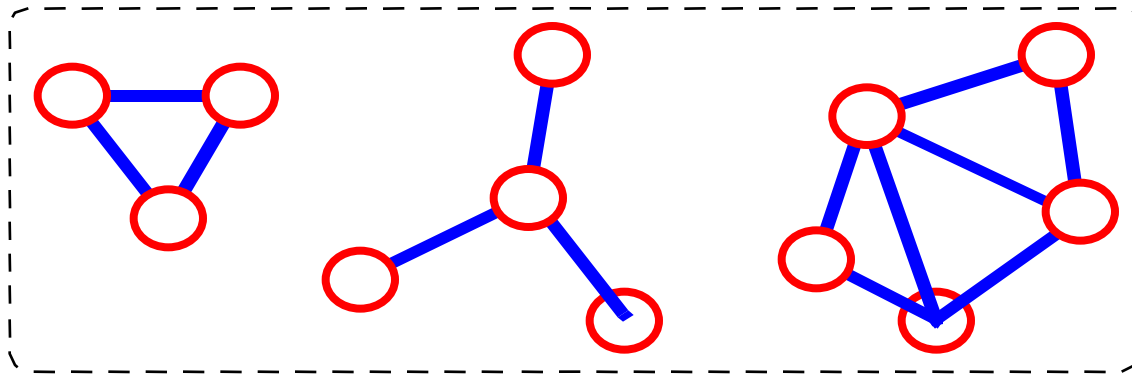


connected



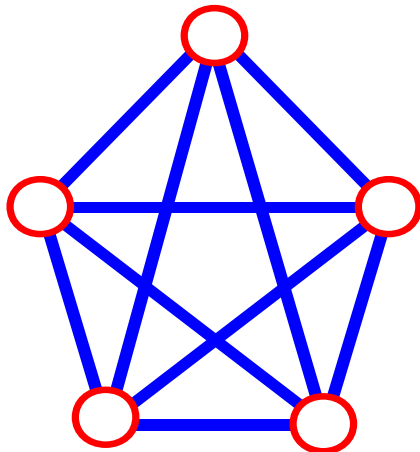
not connected

- **subgraph**: subset of vertices and edges forming a graph
- **connected component**: maximal connected subgraph. E.g., the graph below has 3 connected components.



Connectivity

- Let $n = \text{\#vertices}$, and $m = \text{\#edges}$
- **A complete graph**: one in which all pairs of vertices are adjacent
- *How many total edges in a complete graph?*
 - Each of the n vertices is incident to $n-1$ edges, however, we would have counted each edge twice! Therefore, intuitively, $m = n(n-1)/2$.
- Therefore, if a graph is not complete, $m < n(n-1)/2$



$$n = 5$$

$$m = (5 * 4)/2 = 10$$

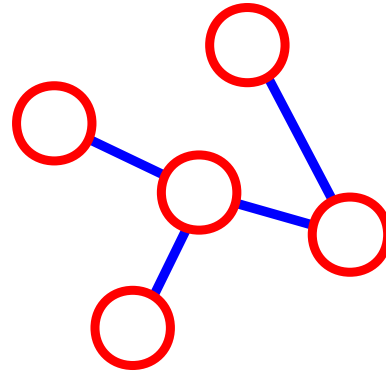
More Connectivity

n = #vertices

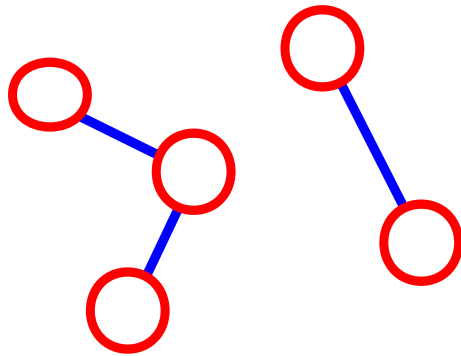
m = #edges

- For a tree **m** = **n** - 1

*If **m** < **n** - 1, G is not connected*

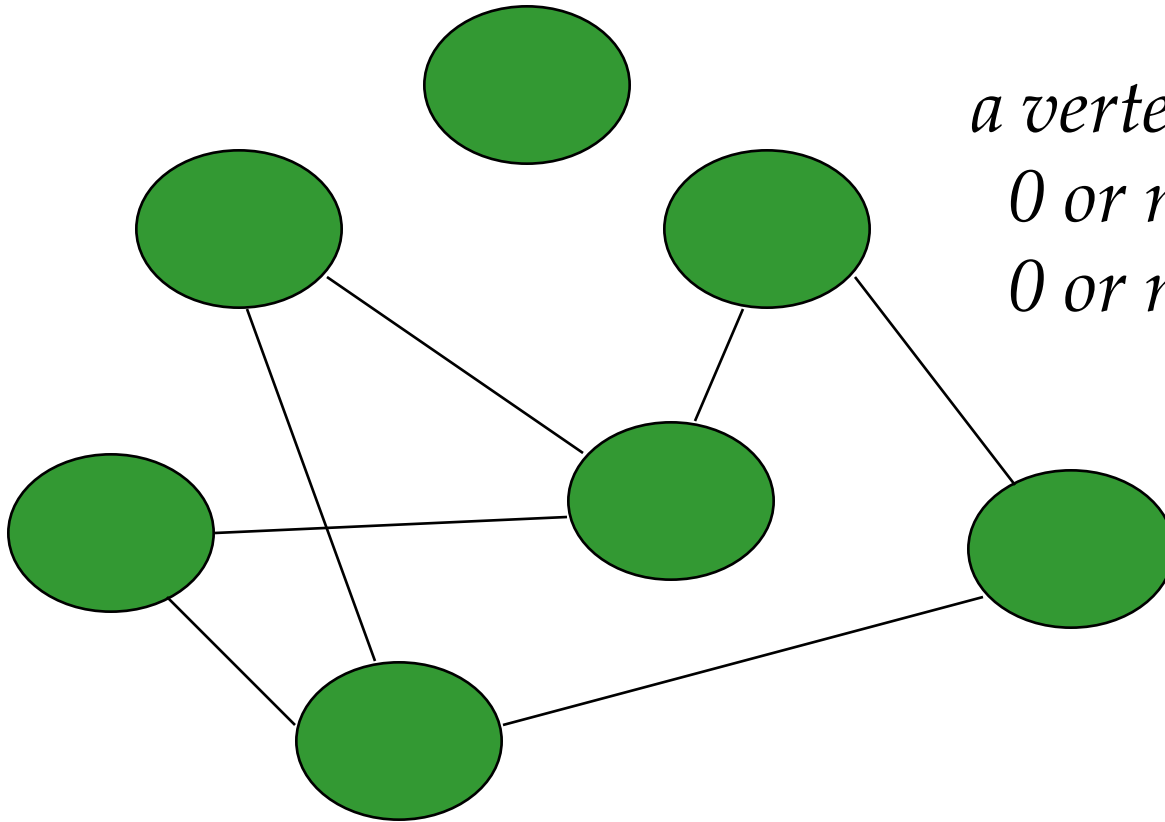


n = 5
m = 4



n = 5
m = 3

$$G = (V, E)$$



*a vertex may have:
0 or more predecessors
0 or more successors*

some problems that can be represented by a graph

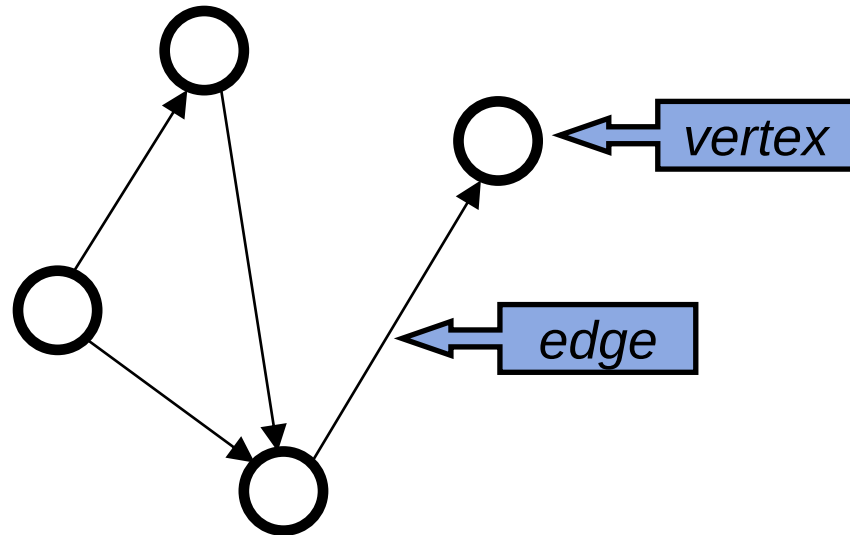
- computer networks
- airline flights
- road map
- course prerequisite structure
- tasks for completing a job
- flow of control through a program
- many more

Graphs in Computer Science: How do they help?

- What do they model?
 - An abstraction in Core CS.
 - *Examples:* VLSI Circuits, Communication Networks, Logical flow of a computer program, Data structures.
 - An abstraction for data and relationships.
 - *Examples:* The Web, Social Networks, Flows and Flow Networks, Biological Data, Taxonomies, Citations, Explicit relations within a DB system.
- What aspects are studied?
 - Algorithms, Data Structures and Complexity Theory.
 - Characterization and Modeling of Graphs.
 - Implementations of Graph Algorithms in Specific contexts.

What is a graph?

- A set of vertices and edges
 - Directed/Undirected
 - Weighted/Unweighted
 - Cyclic/Acyclic



Representation of Graphs

- Adjacency Matrix
 - A $V \times V$ array, with `matrix[i][j]` storing whether there is an edge between the i^{th} vertex and the j^{th} vertex
- Adjacency Linked List
 - One linked list per vertex, each storing directly reachable vertices

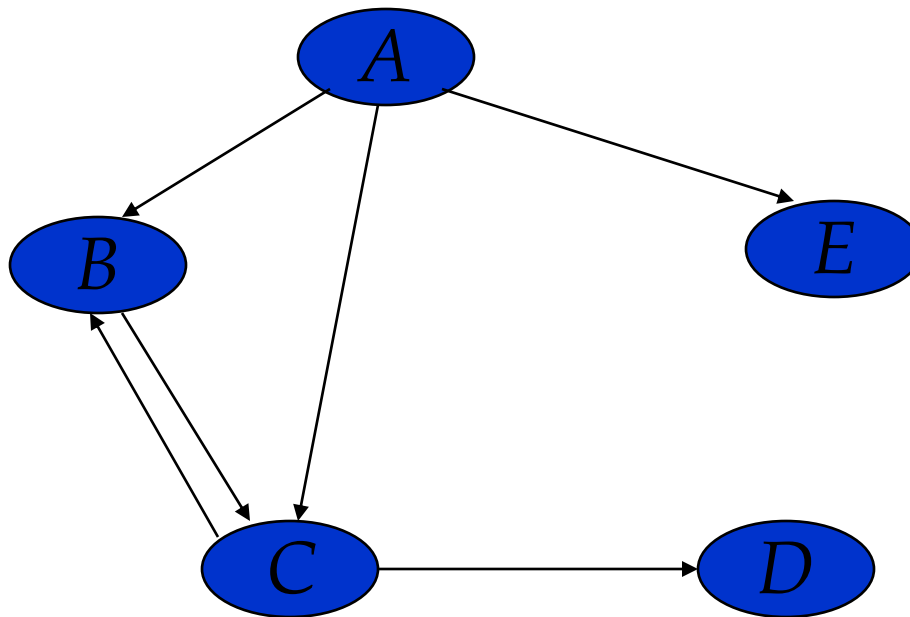
Representation of Graphs

	Adjacency Matrix	Adjacency Linked List	
Memory Storage	$O(V^2)$	$O(V+E)$	
Check whether (u,v) is an edge	$O(1)$	$O(\deg(u))$	
Find all adjacent vertices of a vertex u	$O(V)$	$O(\deg(u))$	

graph variations

- undirected graph (graph)
 - edges do not have a direction
 - $(V1, V2)$ and $(V2, V1)$ are the same edge
- directed graph (digraph)
 - edges have a direction
 - $\langle V1, V2 \rangle$ and $\langle V2, V1 \rangle$ are different edges
- for either type, edges may be weighted or unweighted

a digraph



$V = [A, B, C, D, E]$

$E = [<A,B>, <B,C>, <C,B>, <A,C>, <A,E>, <C,D>]$

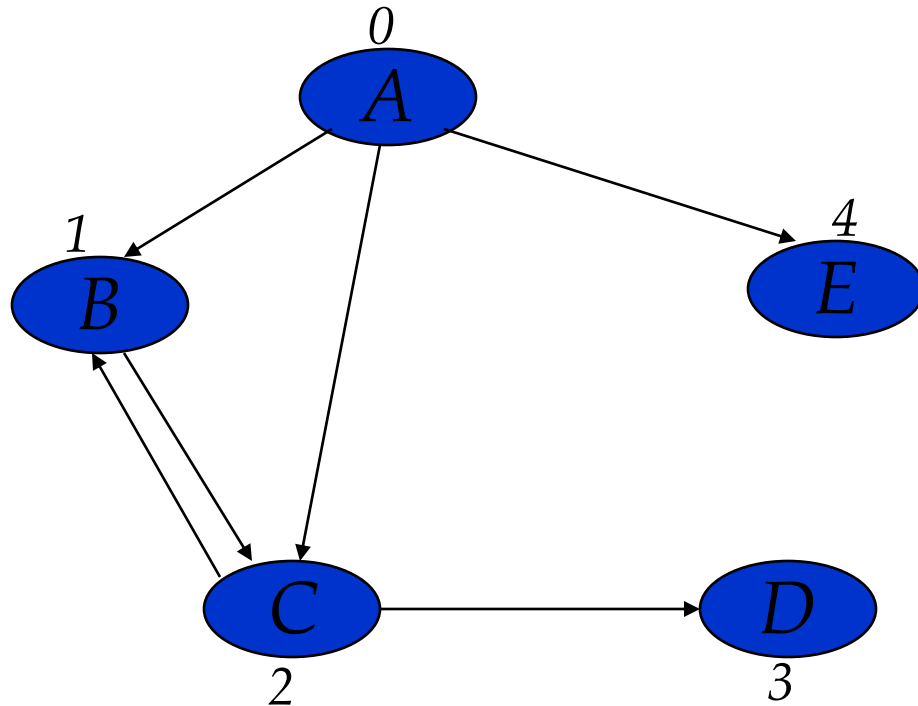
graph data structures

- storing the vertices
 - each vertex has a unique identifier and, maybe, other information
 - for efficiency, associate each vertex with a number that can be used as an index
- storing the edges
 - adjacency matrix – represent all possible edges
 - adjacency lists – represent only the existing edges

storing the vertices

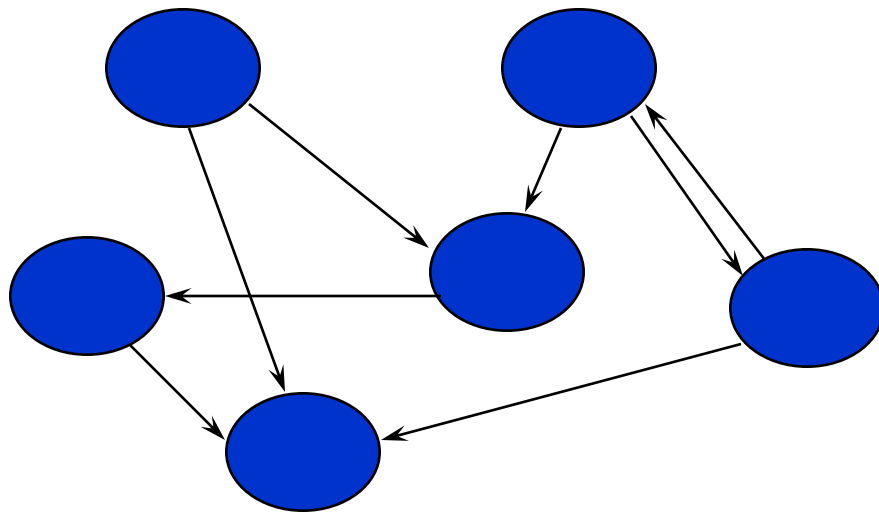
- when a vertex is added to the graph, assign it a number
 - vertices are numbered between 0 and $n-1$
- graph operations start by looking up the number associated with a vertex
- many data structures to use
 - any of the associative data structures
 - for small graphs a vector can be used
 - search will be $O(n)$

the vertex vector



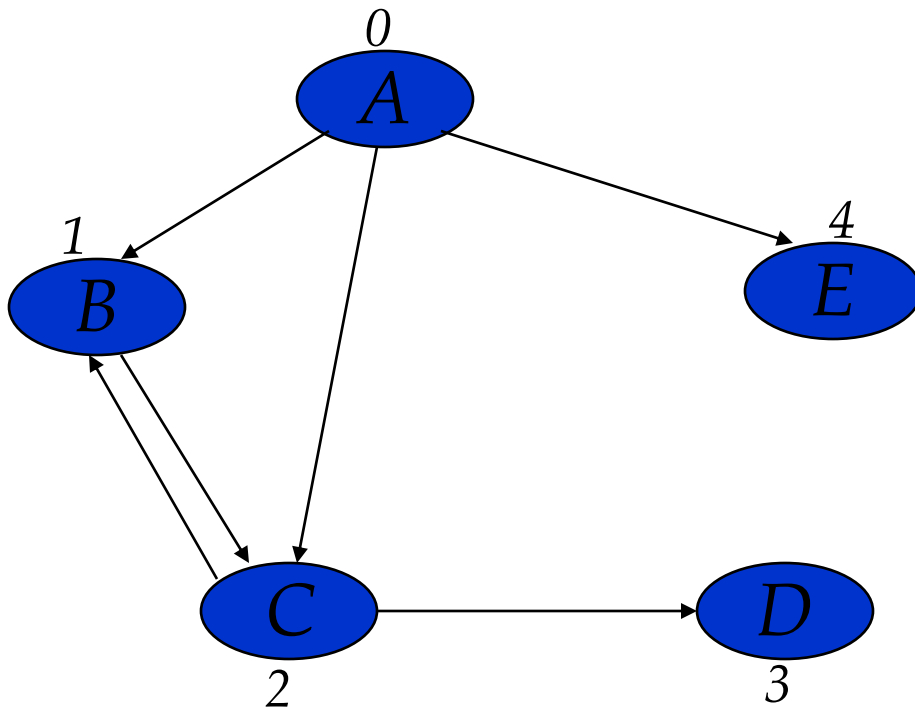
0	A
1	B
2	C
3	D
4	E

maximum # edges?



*a V^2 matrix is needed for
a graph with V vertices*

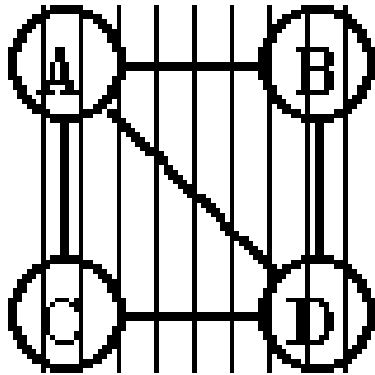
adjacency matrix



0	A
1	B
2	C
3	D
4	E

	0	1	2	3	4
0	0	1	1	0	1
1	0	0	1	0	0
2	0	1	0	1	0
3	0	0	0	0	0
4	0	0	0	0	0

adjacency matrix

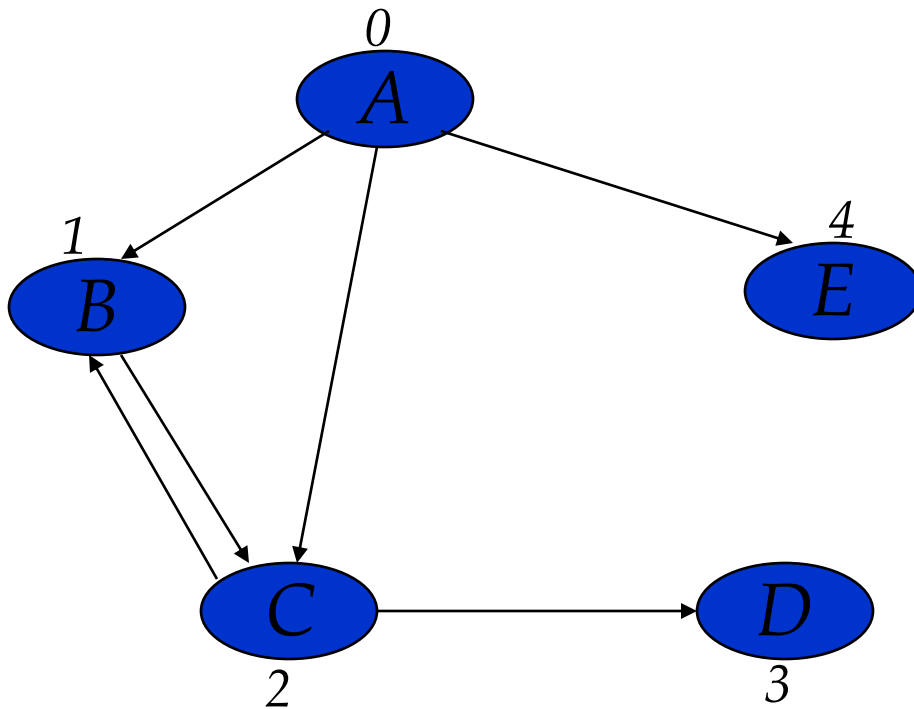


	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>A</i>	0	1	1	1
<i>B</i>	1	0	0	1
<i>C</i>	1	0	0	1
<i>D</i>	1	1	1	0

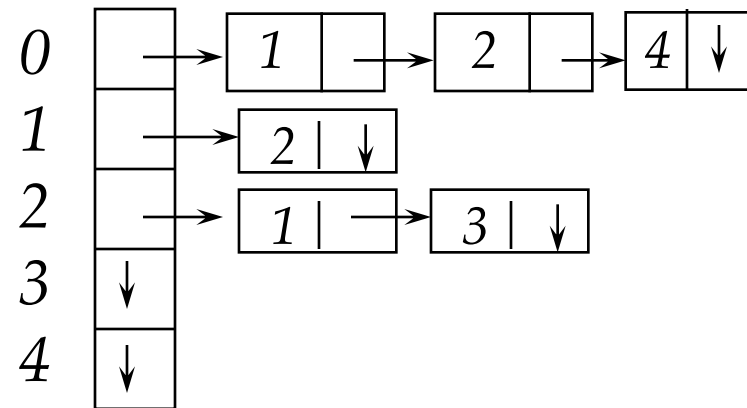
many graphs are “sparse”

- degree of “sparseness” key factor in choosing a data structure for edges
 - adjacency matrix requires space for **all possible** edges
 - adjacency list requires space for existing edges only
- affects amount of memory space needed
- affects efficiency of graph operations

adjacency lists

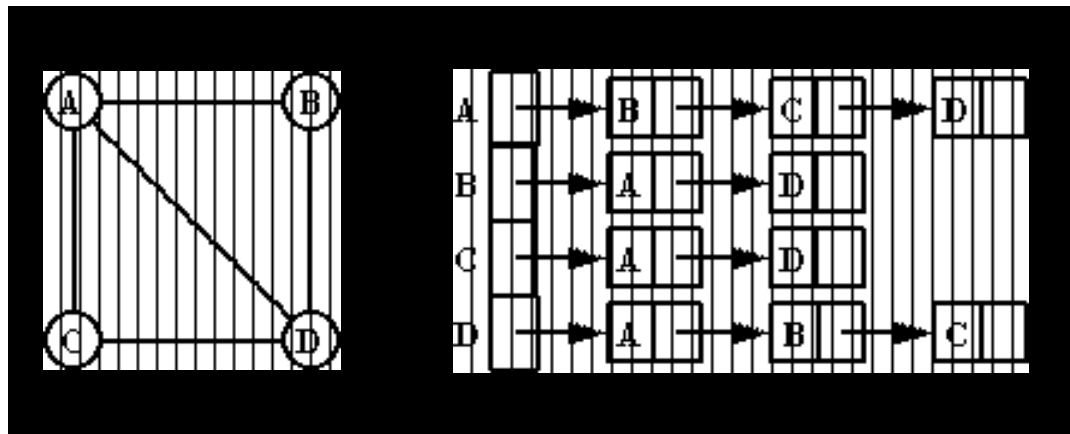


0	A
1	B
2	C
3	D
4	E



Adjacency Lists

A representation of the graph consisting of a list of nodes, with each node containing a list of its neighboring nodes.



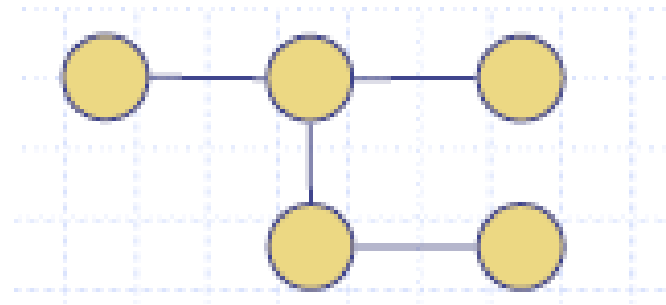
This representation takes $O(|V| + |E|)$ space.

Some graph operations

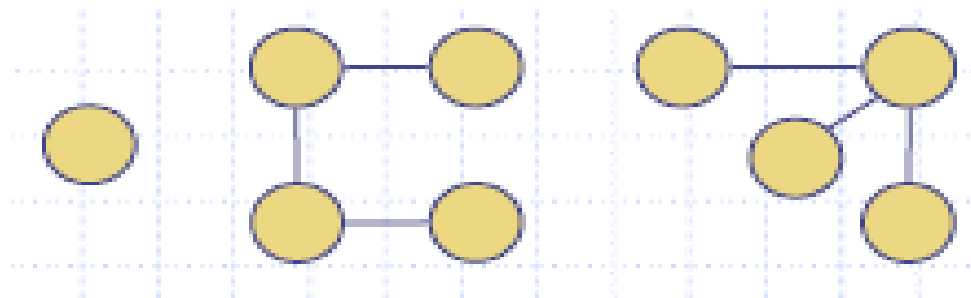
	<u>adjacency matrix</u>	<u>adjacency lists</u>
<i>insertEdge</i>	$O(1)$	$O(e)$
<i>isEdge</i>	$O(1)$	$O(e)$
<i>#successors?</i>	$O(V)$	$O(e)$
<i>#predecessors?</i>	$O(V)$	$O(E)$

Introduction

- A free tree is a connected undirected graph without a cycle.
 - Note: This definition of tree is different from the one of a rooted tree
- In a free tree $|E| = |V| - 1$
- Example of a free tree:

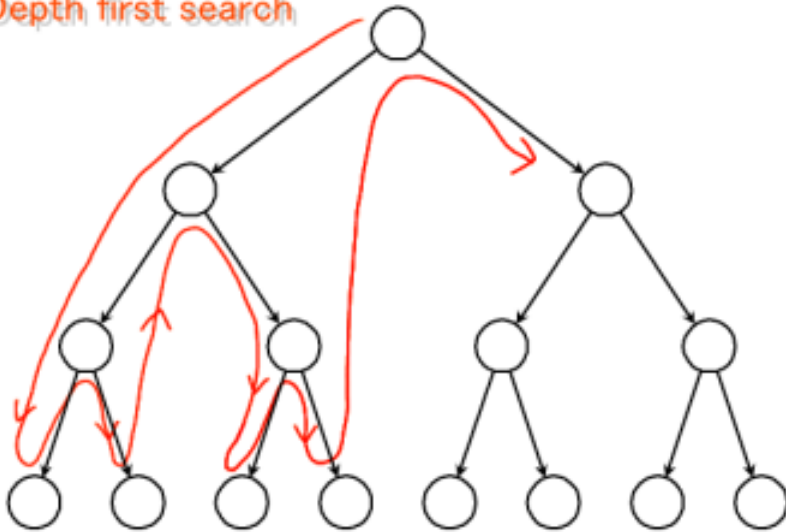


- A forest is an acyclic directed or undirected graph consisting of two or more trees
- The trees in a directed forest are rooted trees
- The trees in an undirected forest are free trees

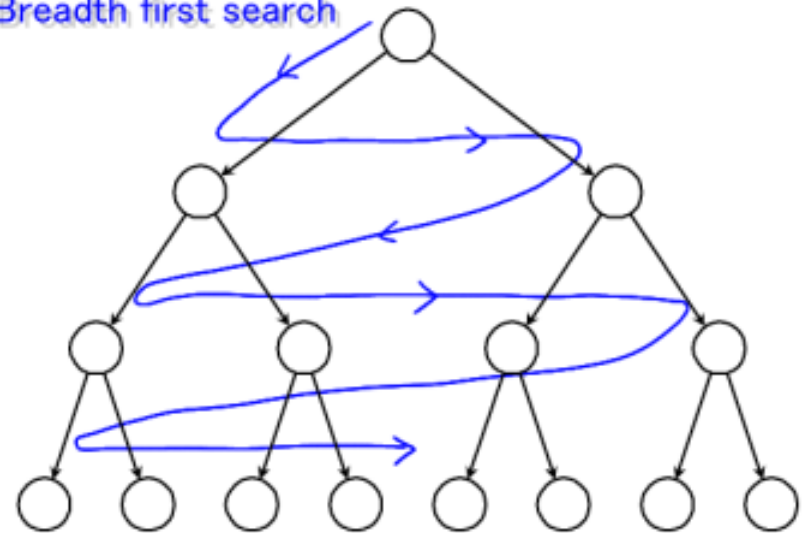


Graph Traversals

Depth first search



Breadth first search



• Both take time: $O(V+E)$

Use of a stack

- It is very common to use a stack to keep track of:
 - nodes to be visited next, or
 - nodes that we have already visited.
- Typically, use of a stack leads to a *depth-first* visit order.
- Depth-first visit order is “aggressive” in the sense that it examines complete paths.

Use of a queue

- It is very common to use a queue to keep track of:
 - nodes to be visited next, or
 - nodes that we have already visited.
- Typically, use of a queue leads to a *breadth-first* visit order.
- Breadth-first visit order is “cautious” in the sense that it examines every path of length i before going on to paths of length $i+1$.

Breadth-First Search (BFS)

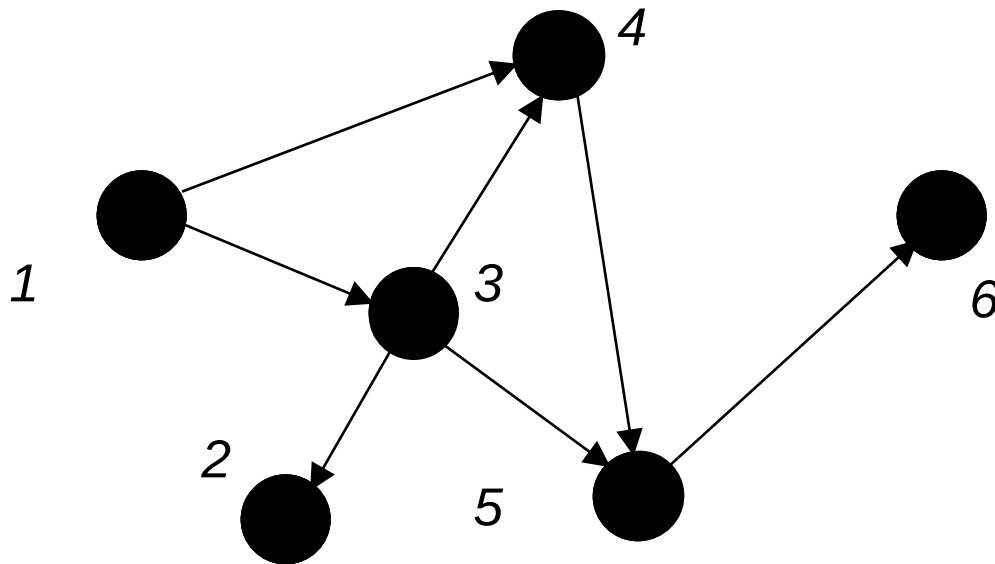
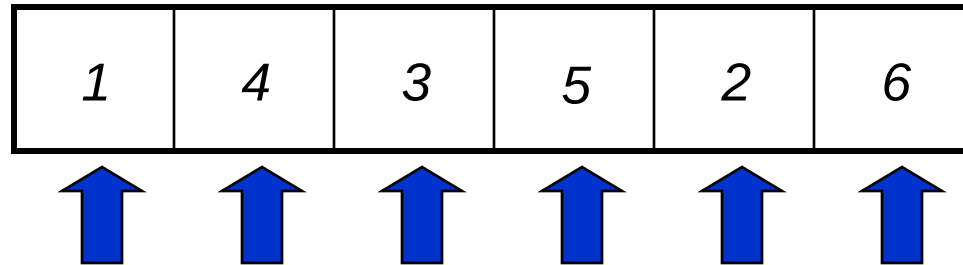
- BFS tries to search all paths.
- BFS makes use of a queue to store visited (but not dead) vertices, expanding the path from the earliest visited vertices.

BFS(G, s)

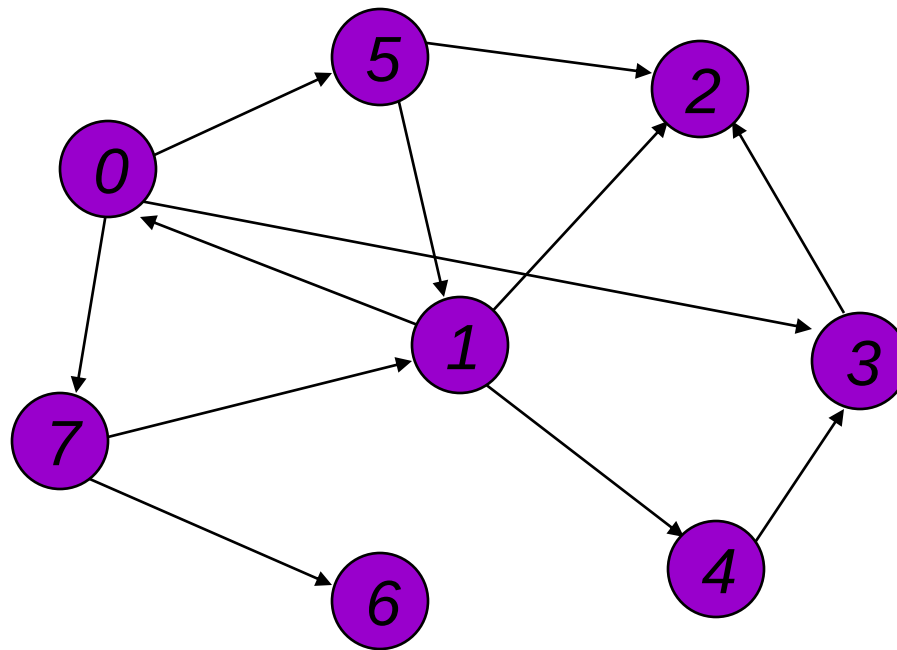
```
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 
```

Simulation of BFS

- Queue:

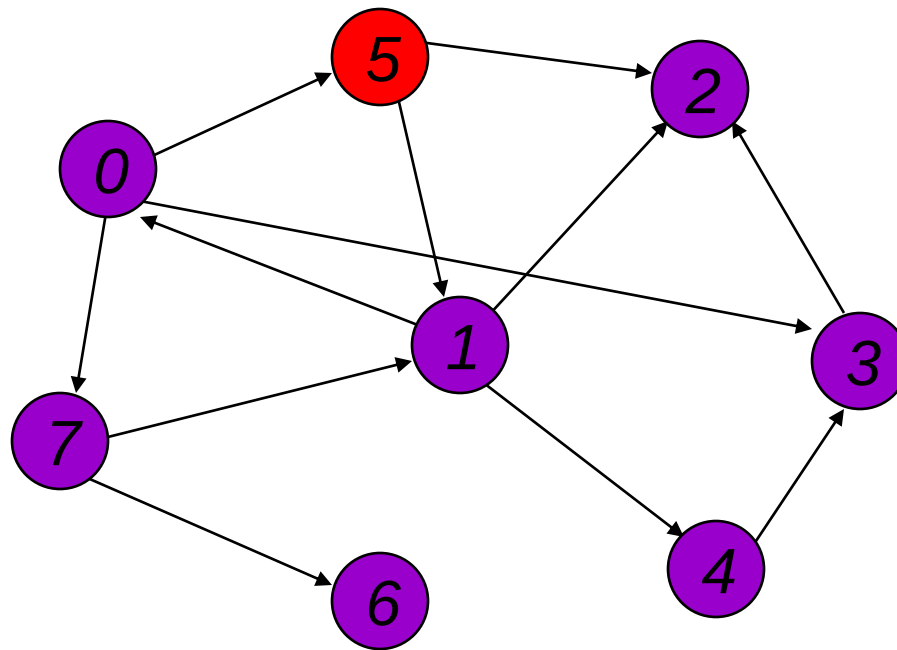


BFS: Start with Node 5



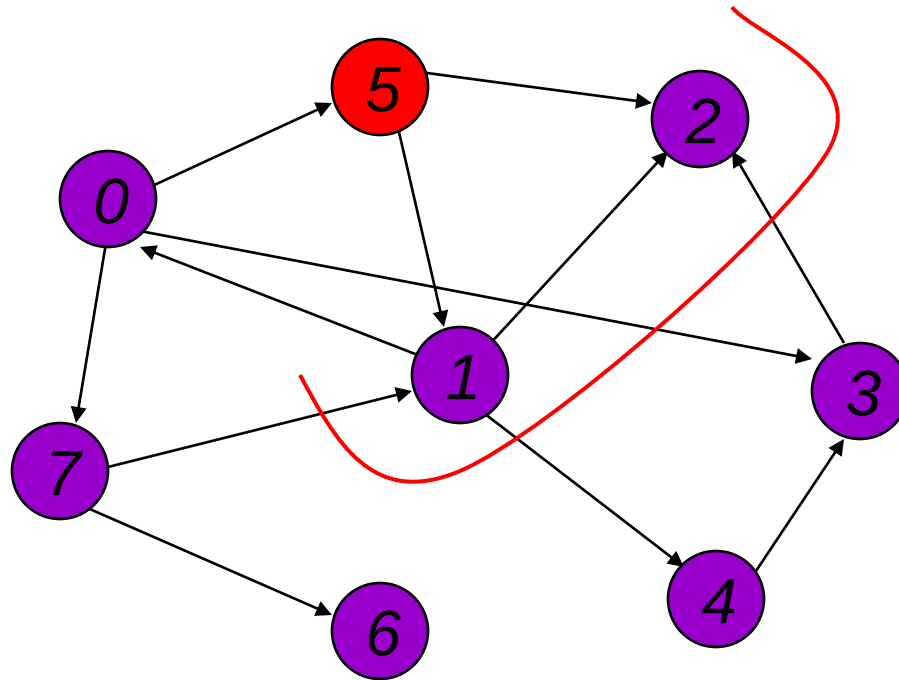
5 1 2 0 4 3 7 6

BFS: Start with Node 5

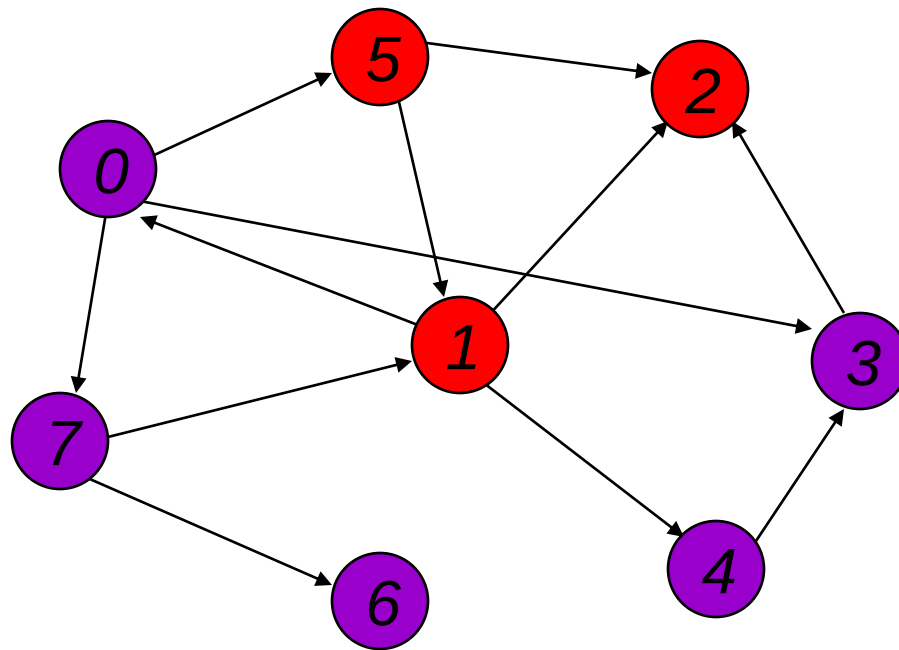


5

BFS: Node one-away

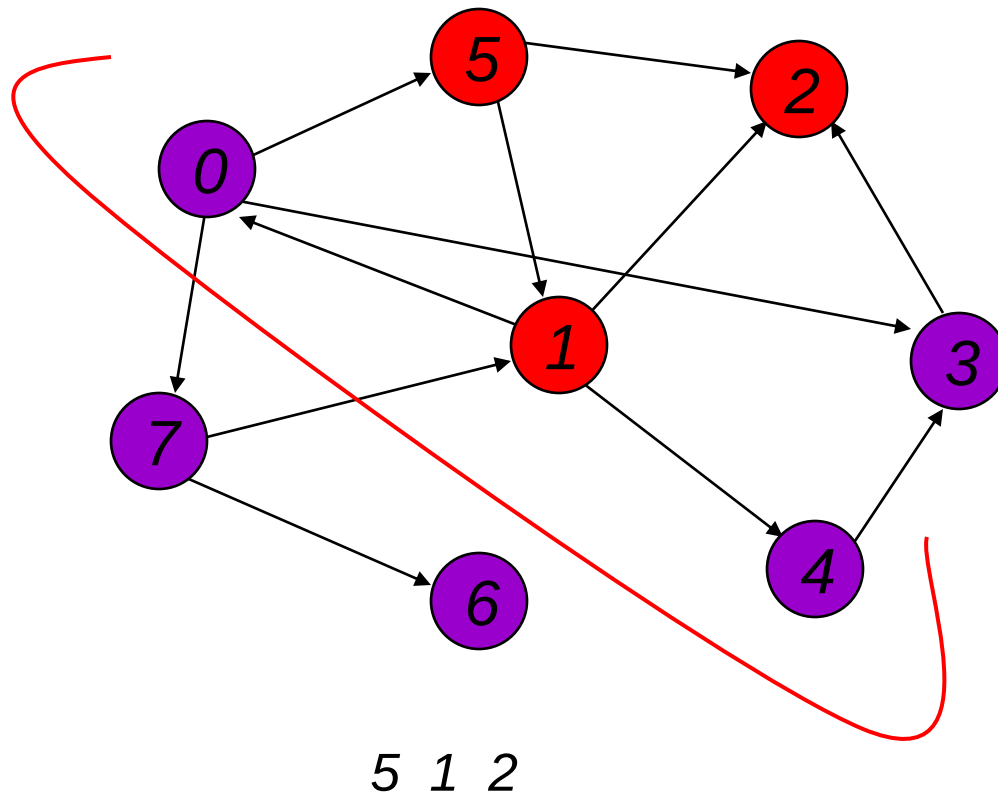


BFS: Visit 1 and 2

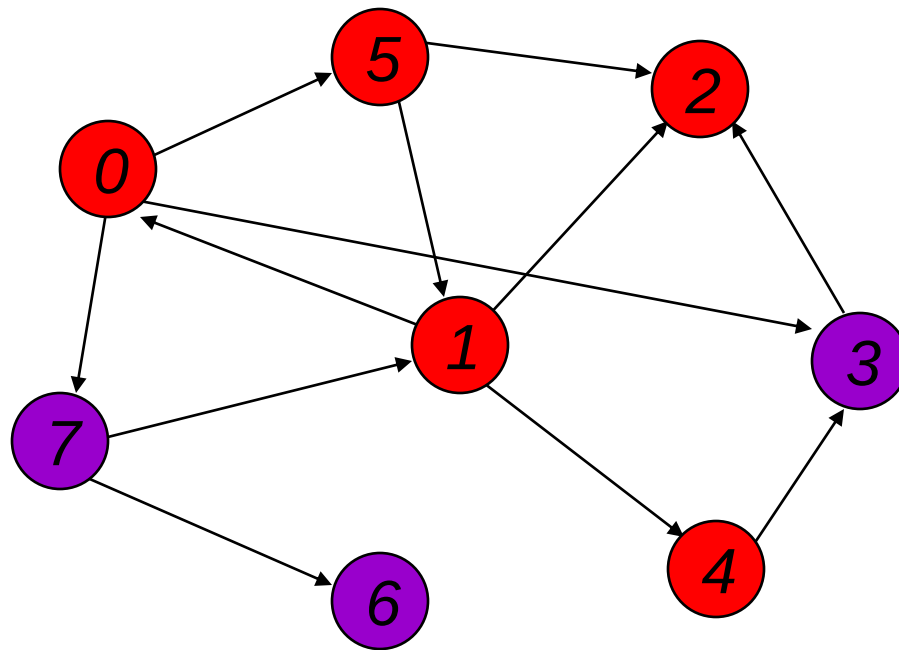


5 1 2

BFS: Nodes two-away

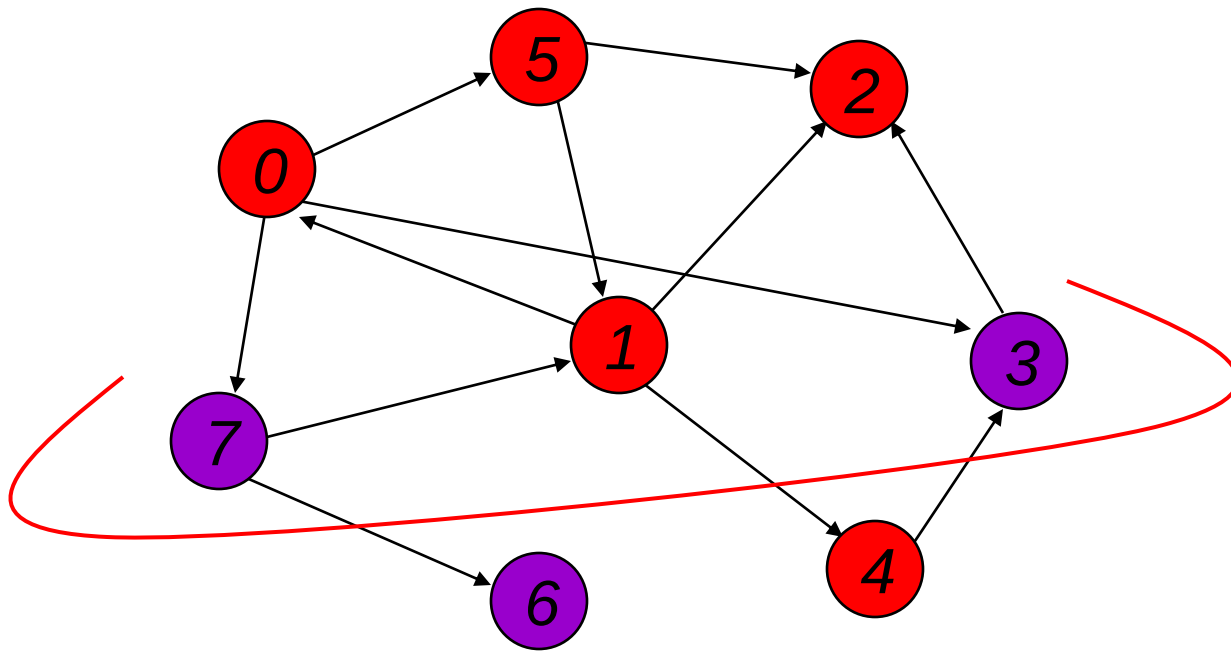


BFS: Visit 0 and 4



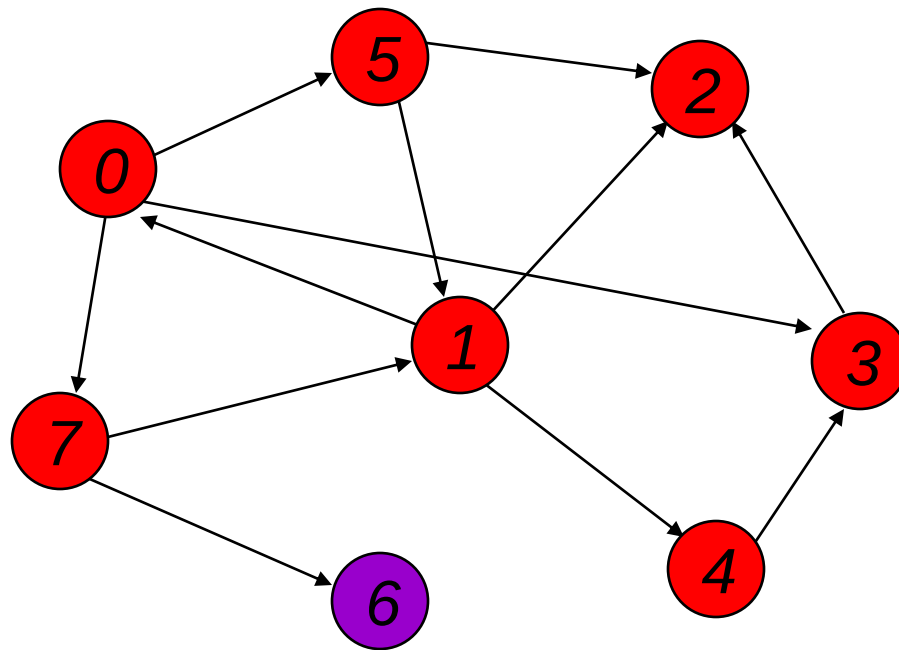
5 1 2 0 4

BFS: Nodes three-away



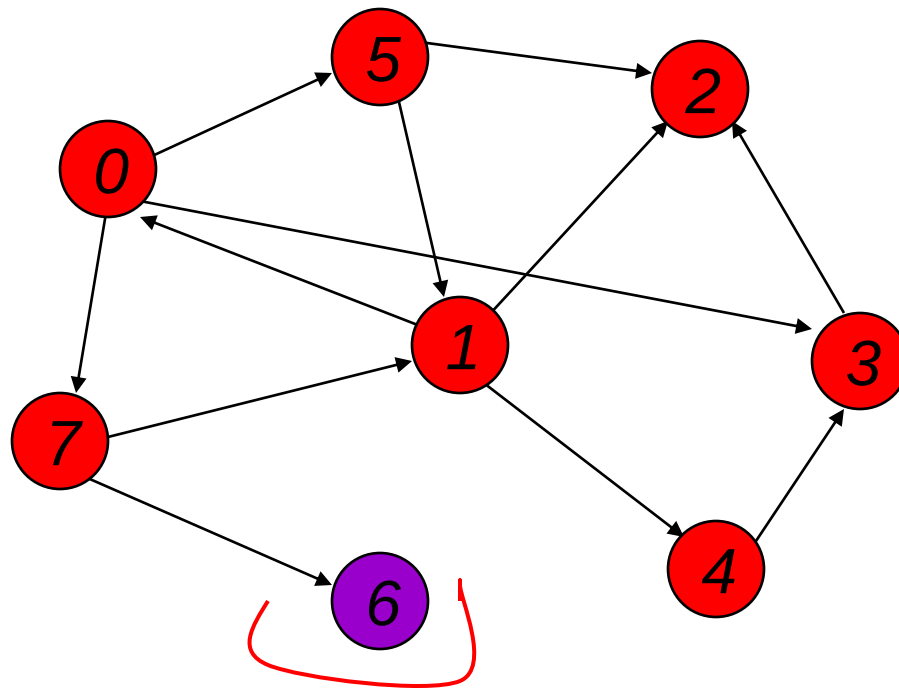
5 1 2 0 4

BFS: Visit nodes 3 and 7



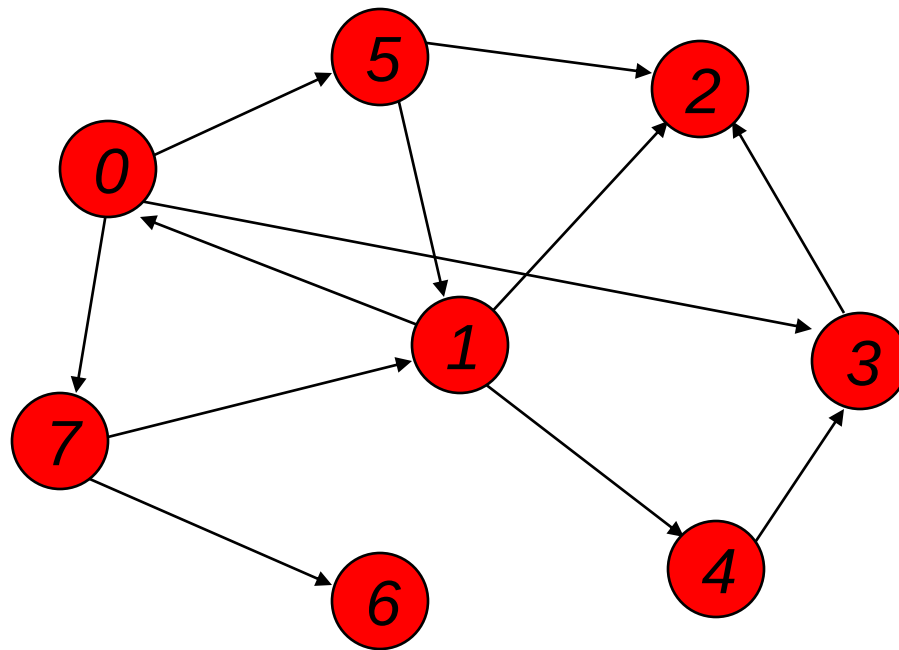
5 1 2 0 4 3 7

BFS: Node four-away



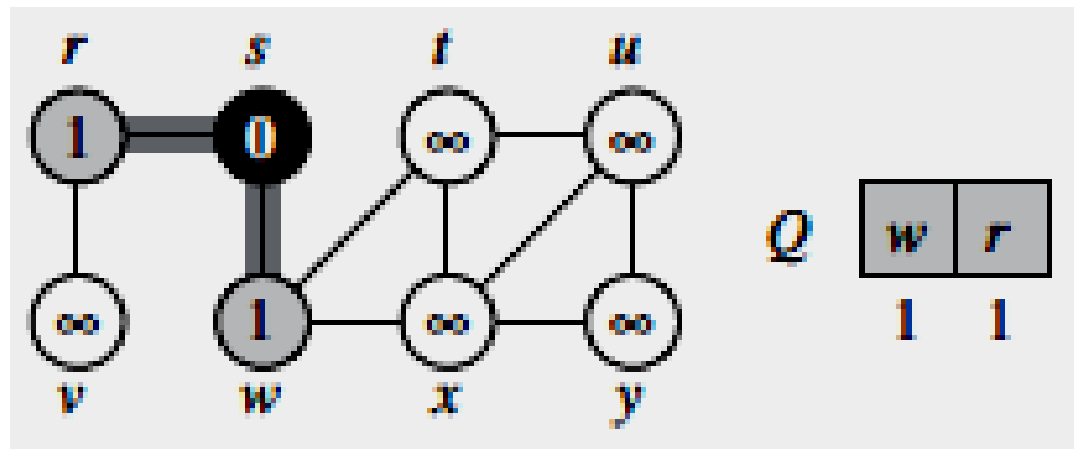
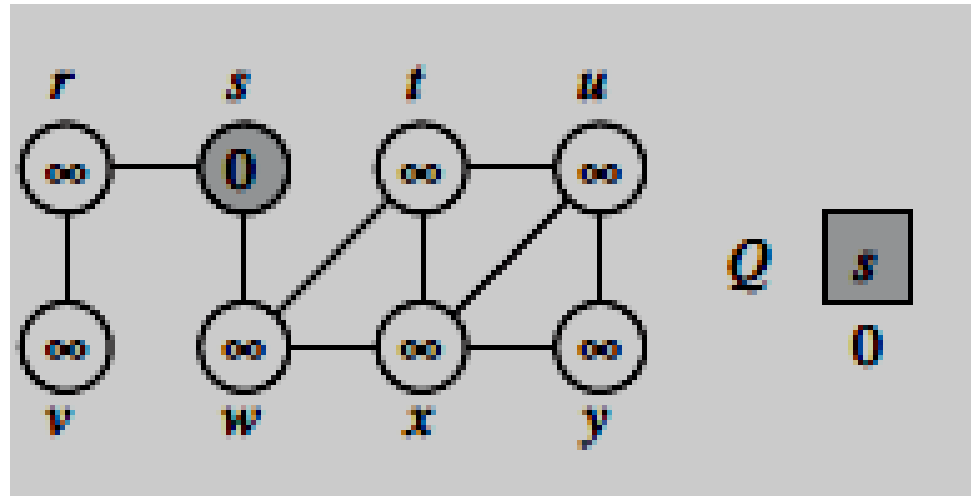
5 1 2 0 4 3 7

BFS: Visit 6

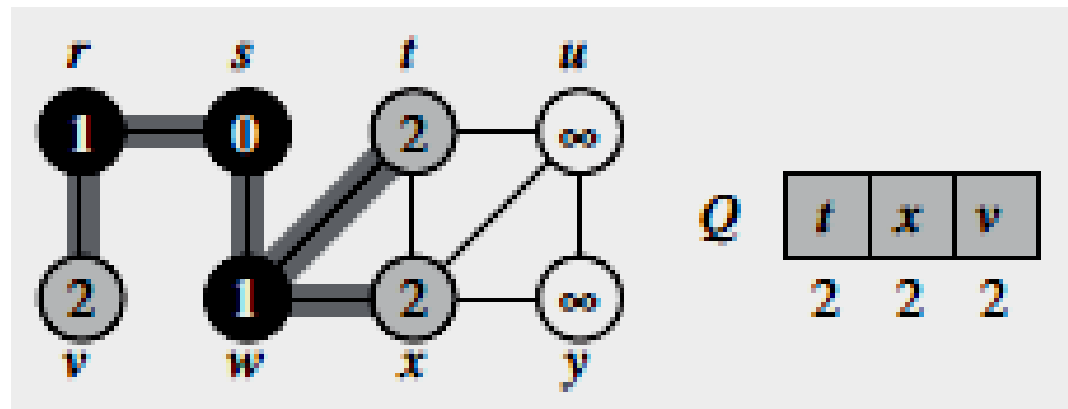
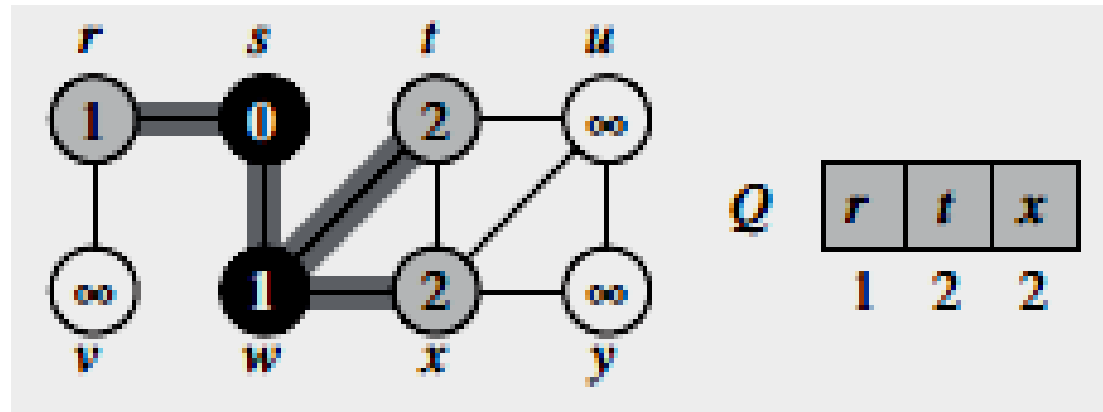


5 1 2 0 4 3 7 6

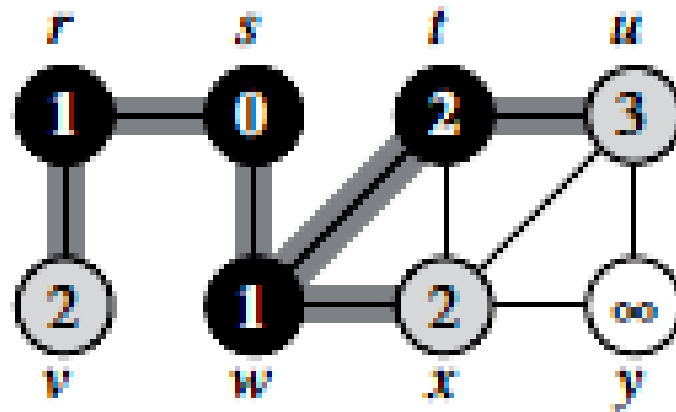
Breadth-First Search (BFS)



Breadth-First Search (BFS)

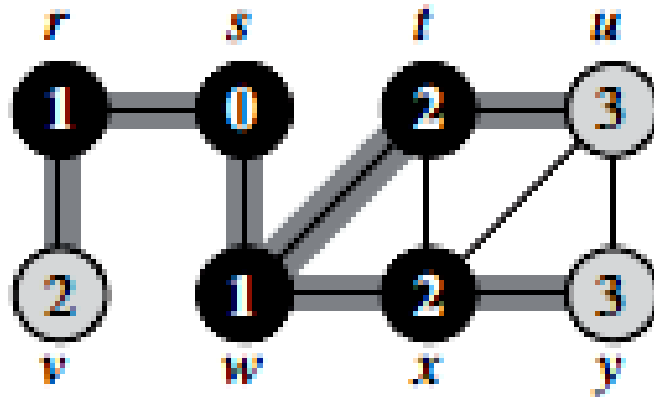


Breadth-First Search (BFS)



Q

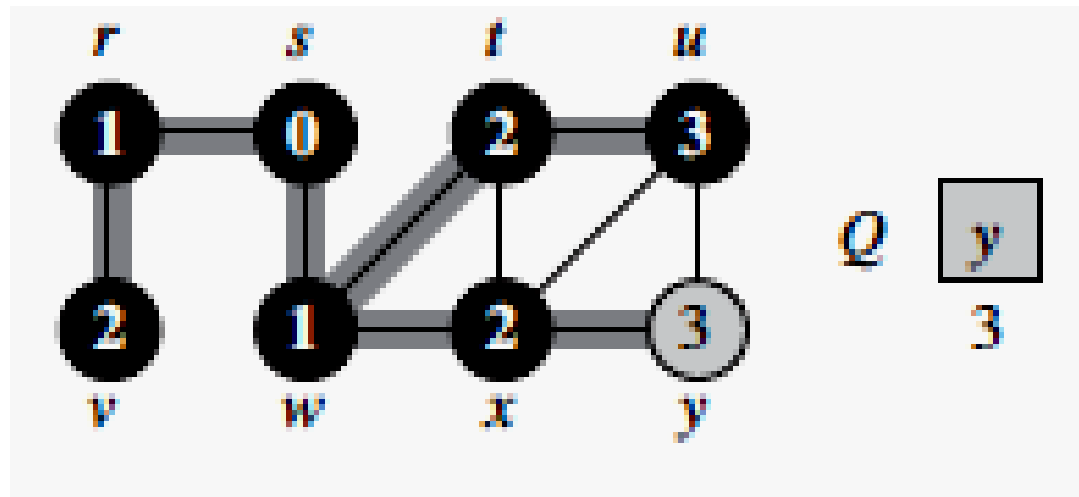
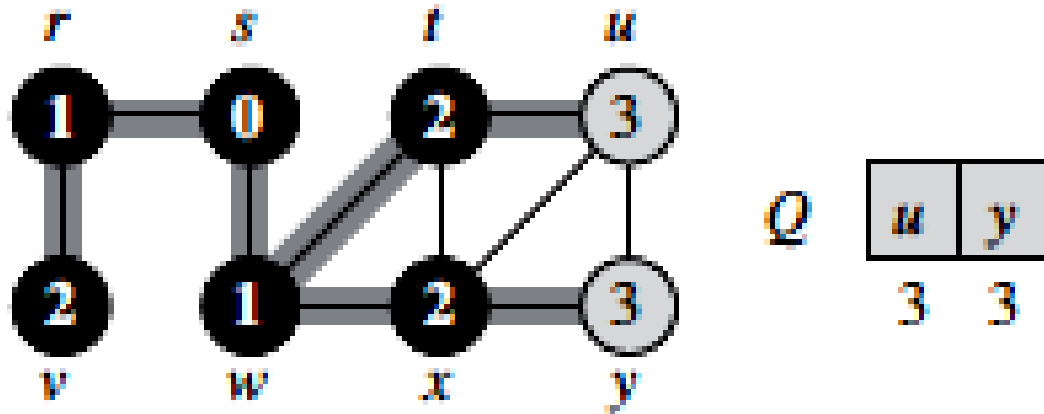
x	v	u
2	2	3



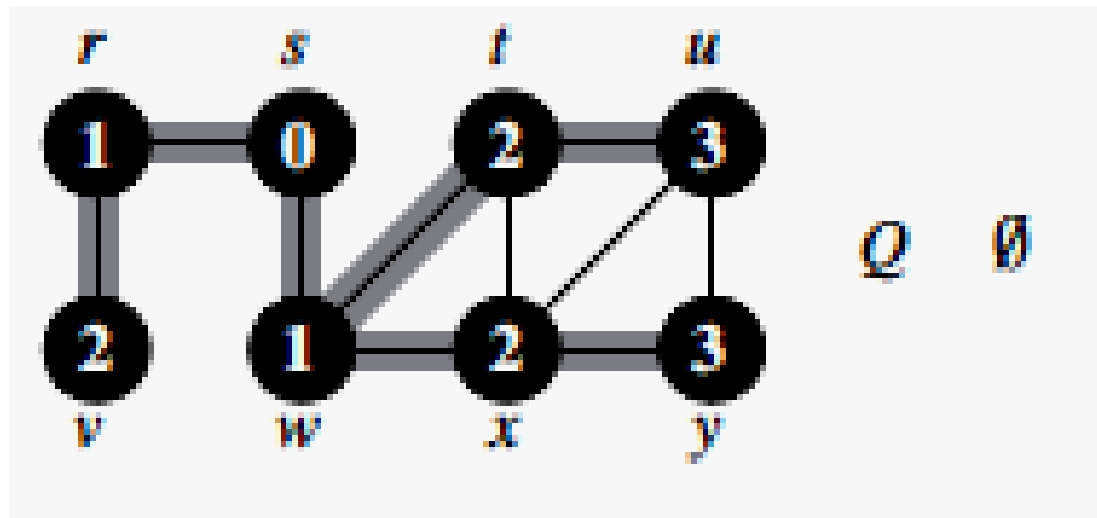
Q

v	u	y
2	3	3

Breadth-First Search (BFS)

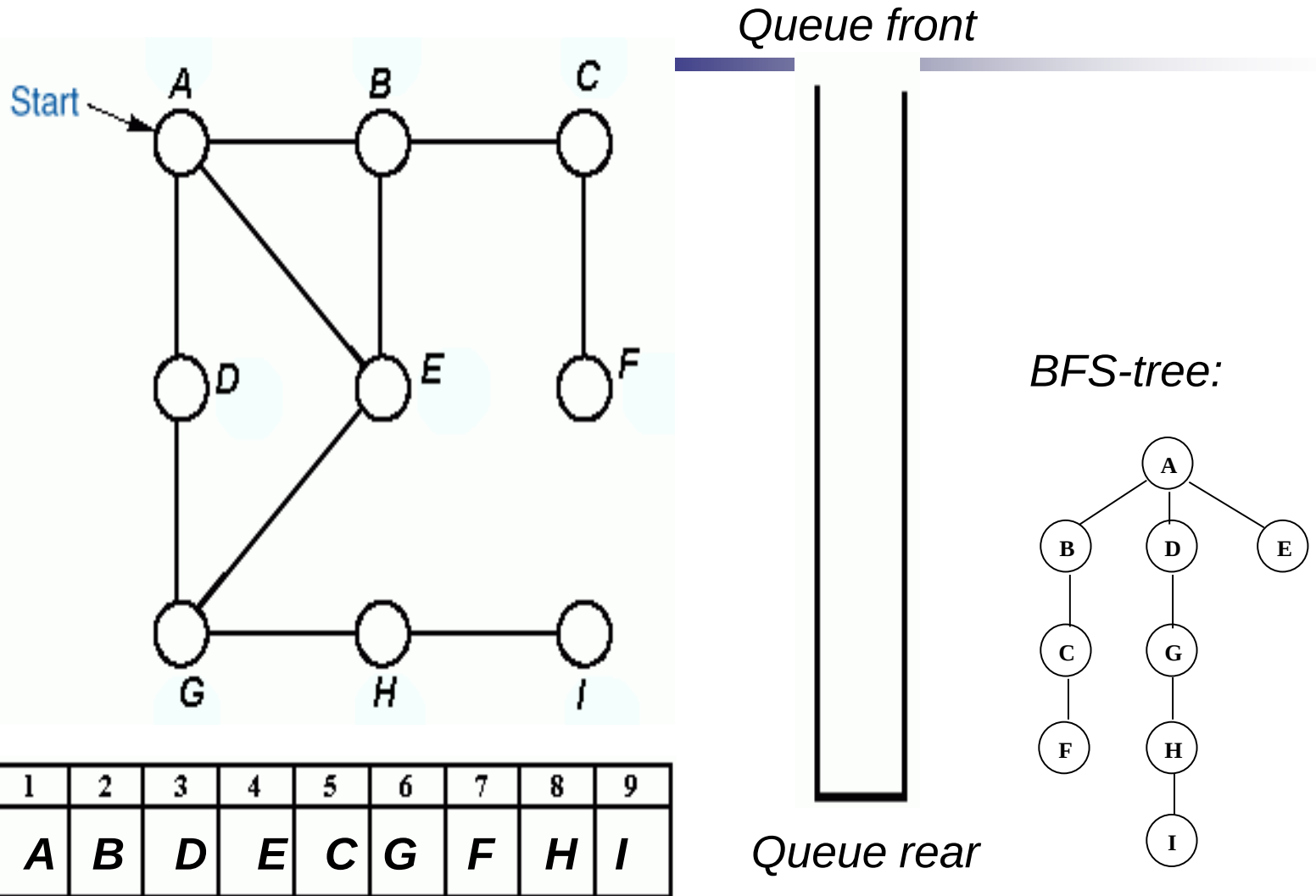


Breadth-First Search (BFS)



Example

- Breadth-first traversal using a queue.



Note: The BFS-tree for undirected graph is a free tree

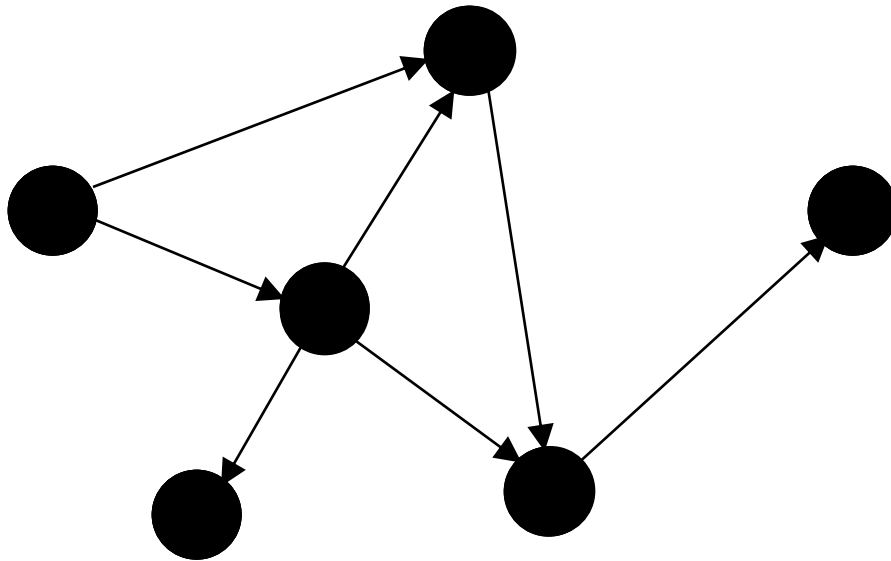
Analysis of BFS

For a Graph $G=(V, E)$ and $n = |V|$ and $m=|E|$

- *When **Adjacency List** is used*
 - *Complexity is $O(m + n)$*
- *When **Adjacency Matrix** is used*
 - *Scanning each row for checking the connectivity of a Vertex is in order $O(n)$.*
 - *So, Complexity is $O(n^2)$*

Depth-First Search (DFS)

- Strategy: Go as far as you can (if you have not visit there), otherwise, go back and try another way



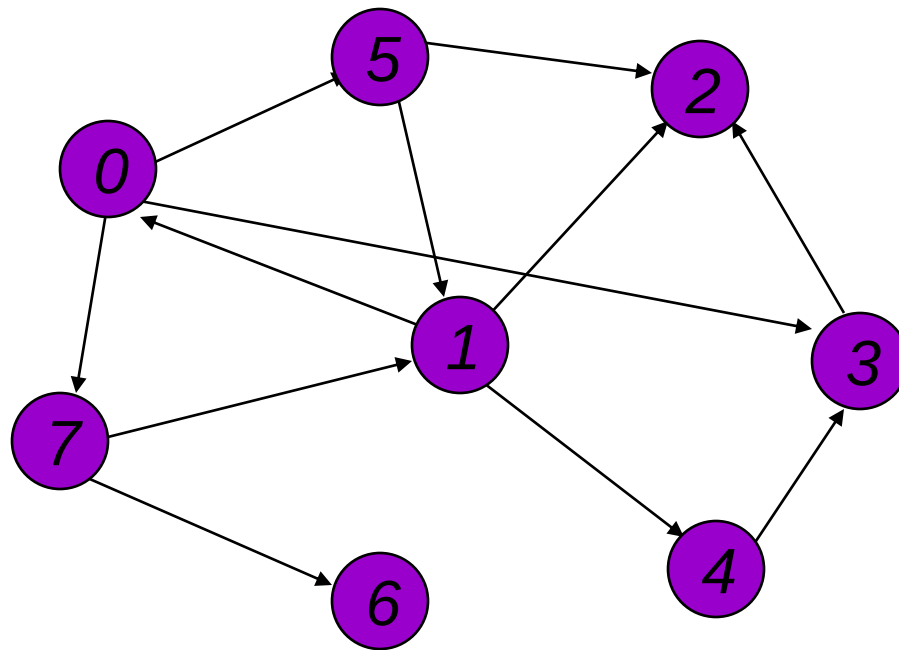
DFS(G)

```
1  for each vertex  $u \in G.V$ 
2       $u.color = WHITE$ 
3       $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == WHITE$ 
7          DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

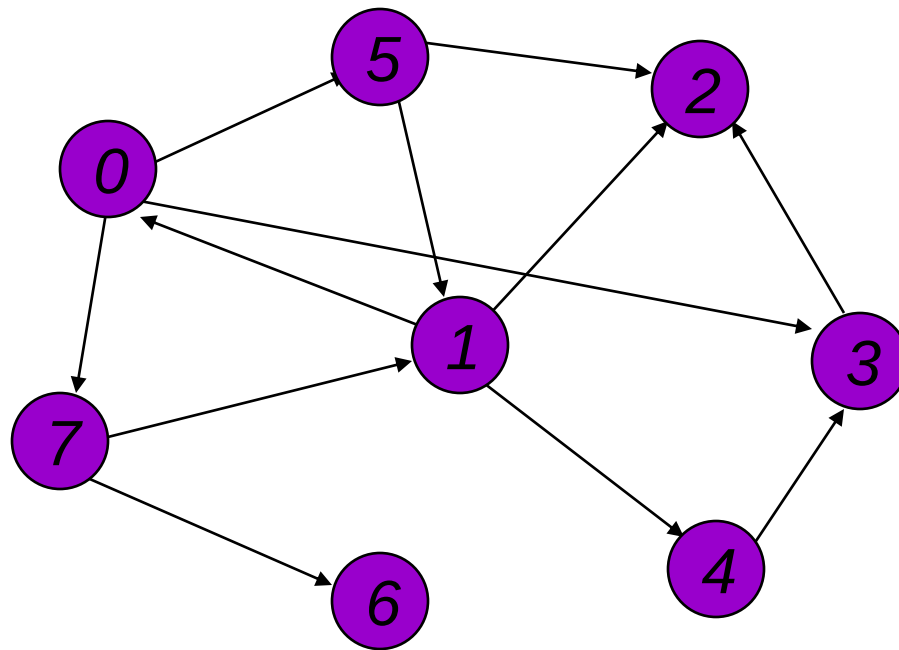
```
1   $time = time + 1$                                 // white vertex  $u$  has just been discovered
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$                             // explore edge  $(u, v)$ 
5      if  $v.color == WHITE$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = BLACK$                                 // blacken  $u$ ; it is finished
9   $time = time + 1$ 
10  $u.f = time$ 
```

Example



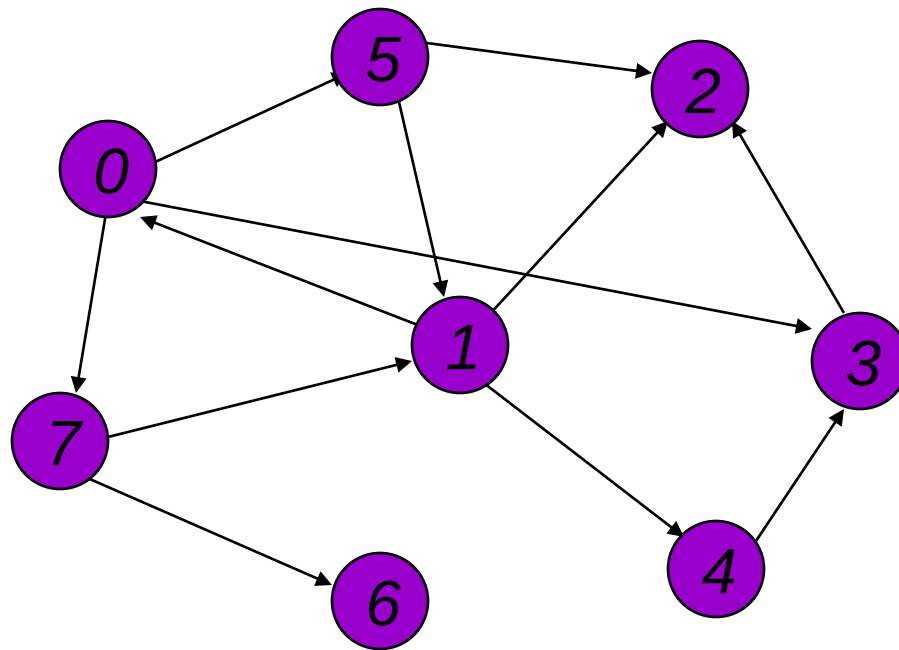
Policy: Visit adjacent nodes in increasing index order

Preorder DFS: Start with Node 5



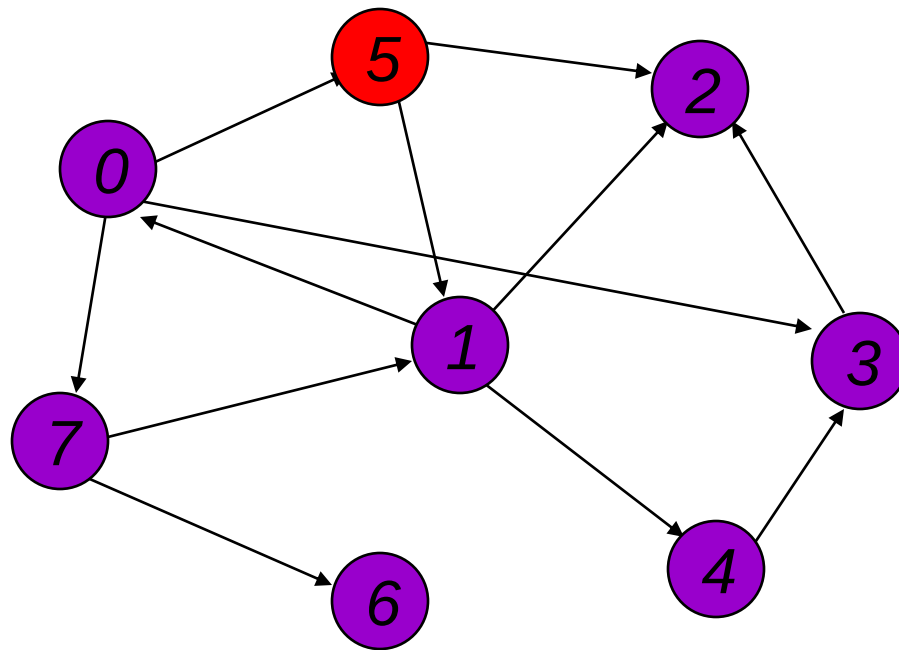
5 1 0 3 2 7 6 4

Preorder DFS: Start with Node 5



Push 5

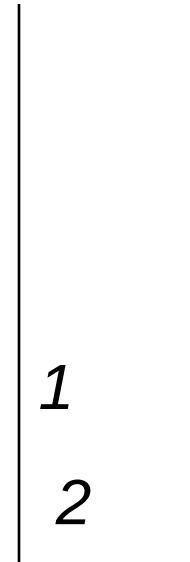
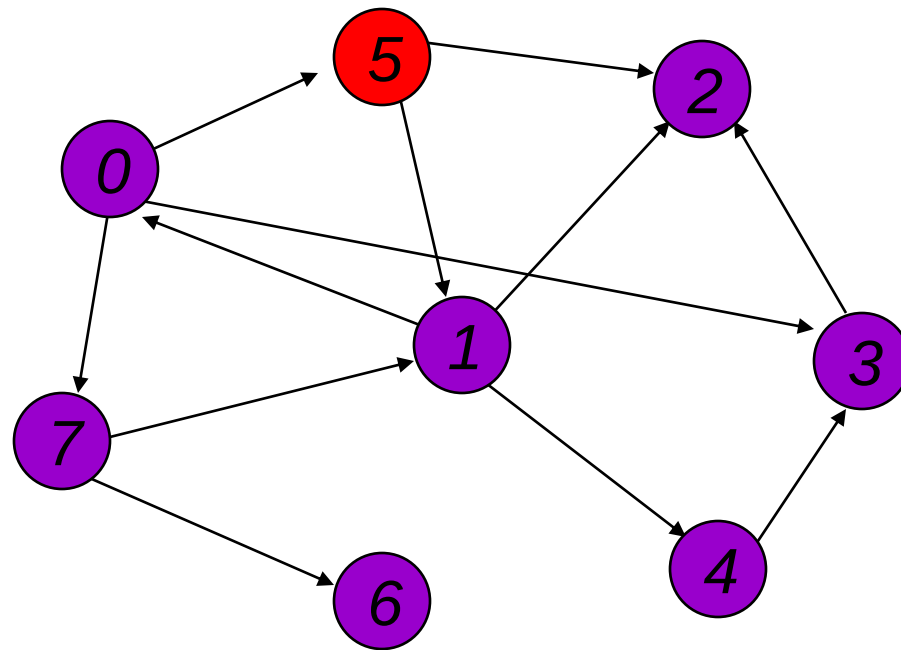
Preorder DFS: Start with Node 5



Pop/Visit/Mark 5

5

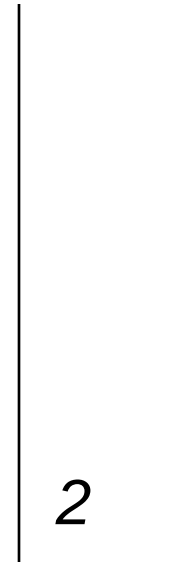
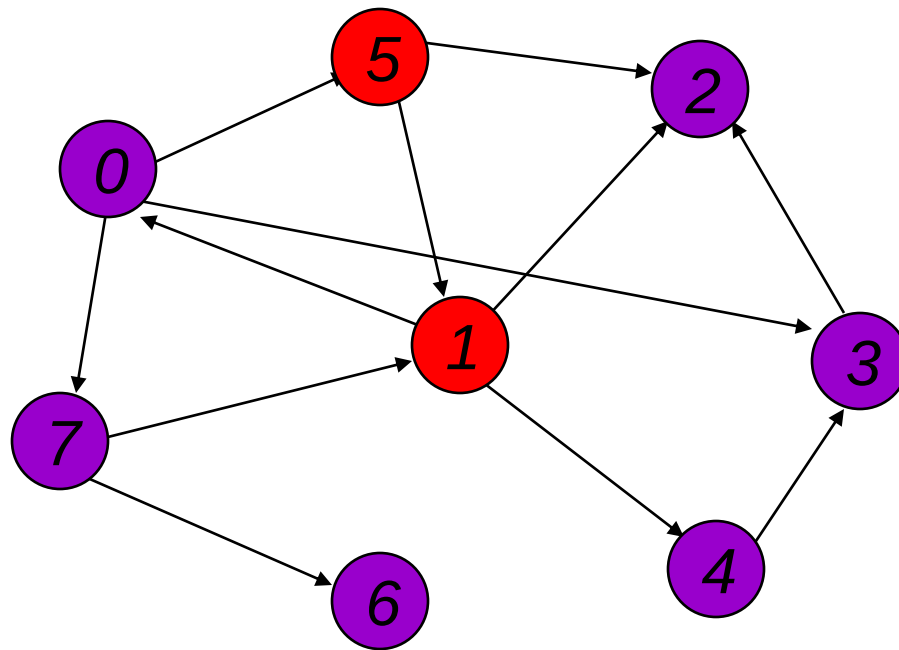
Preorder DFS: Start with Node 5



Push 2, Push 1

5

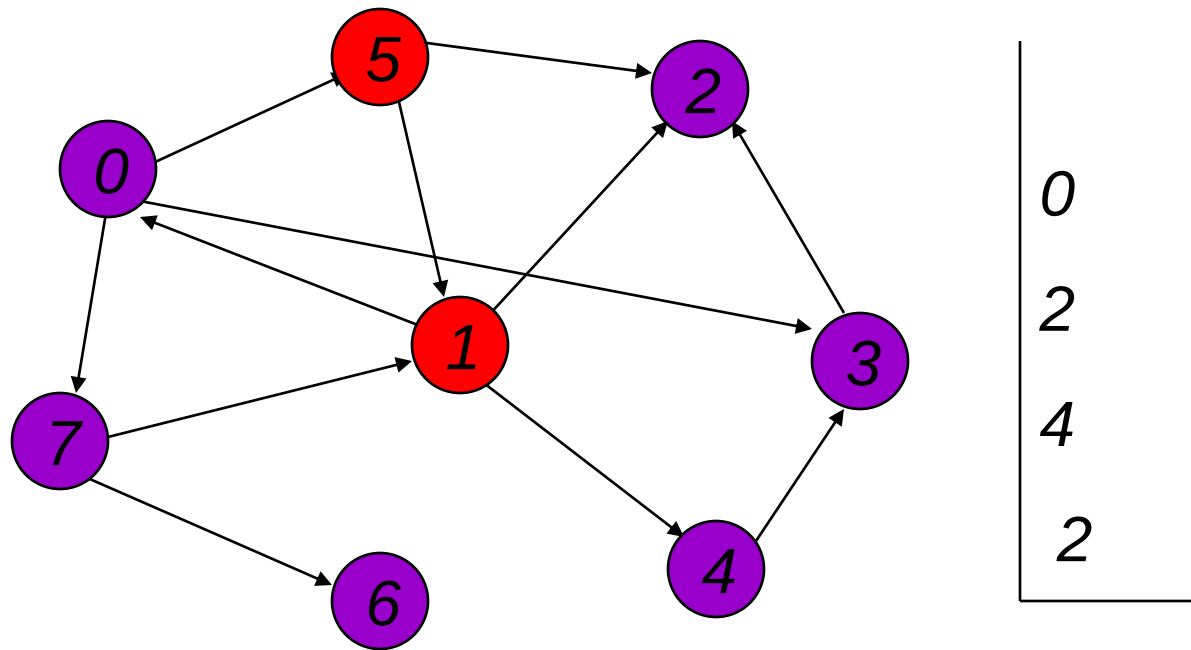
Preorder DFS: Start with Node 5



Pop/Visit/Mark 1

5 1

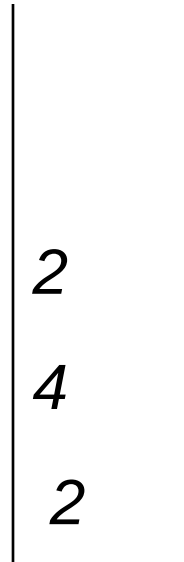
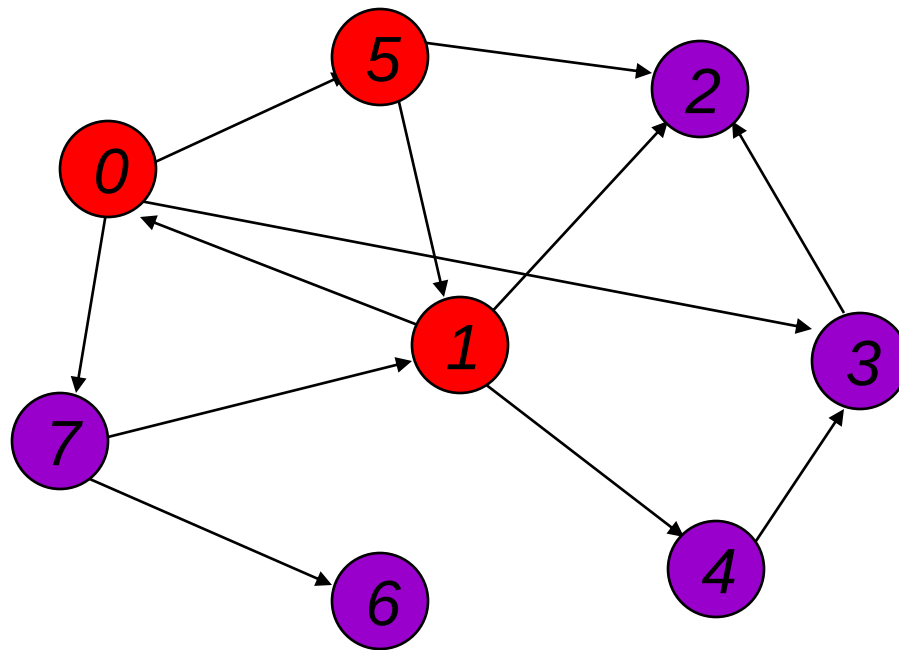
Preorder DFS: Start with Node 5



Push 4, Push 2,
Push 0

5 1

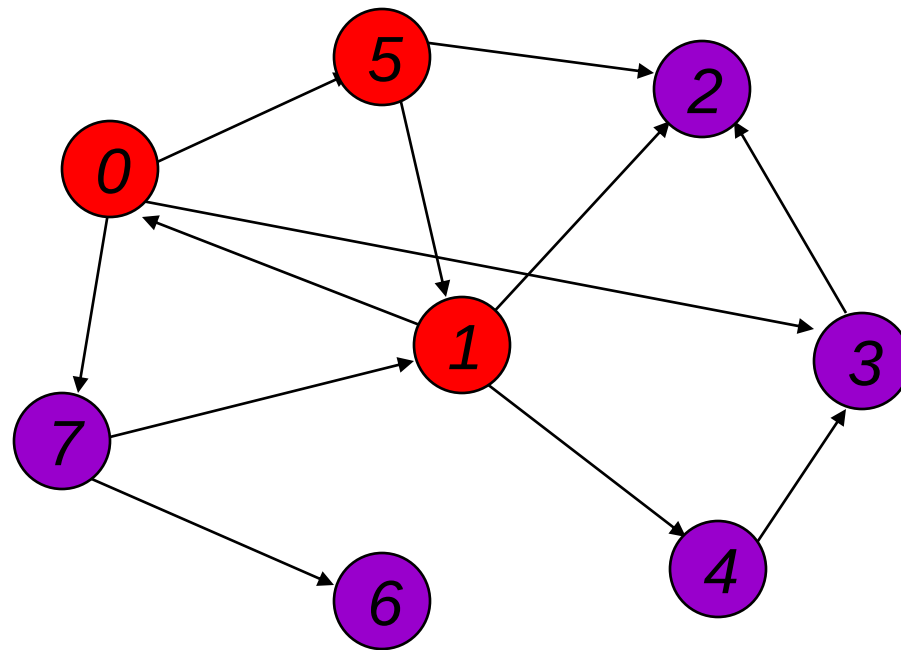
Preorder DFS: Start with Node 5



Pop/Visit/Mark 0

5 1 0

Preorder DFS: Start with Node 5

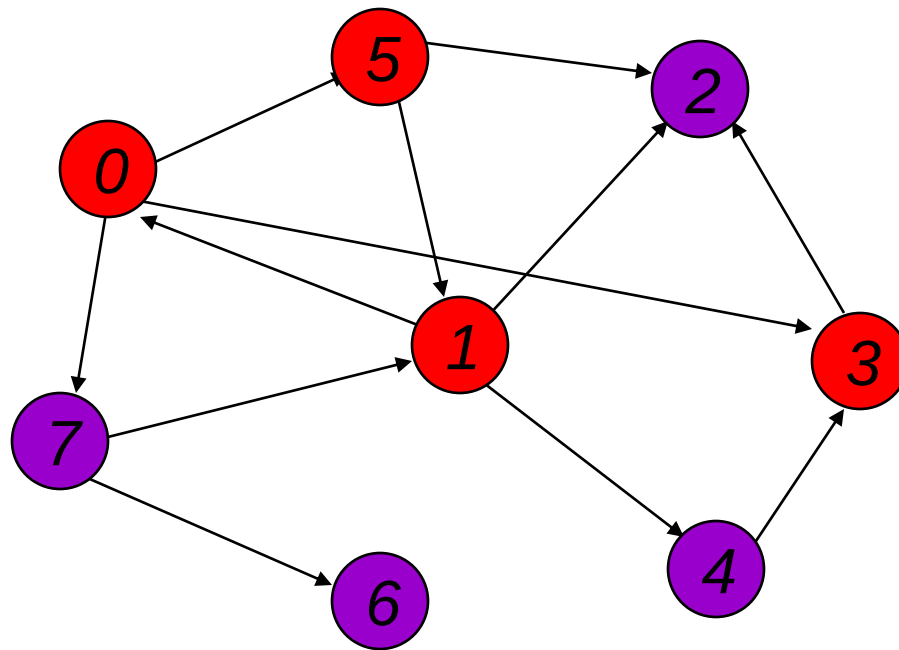


3
7
2
4
2

Push 7, Push 3

5 1 0

Preorder DFS: Start with Node 5

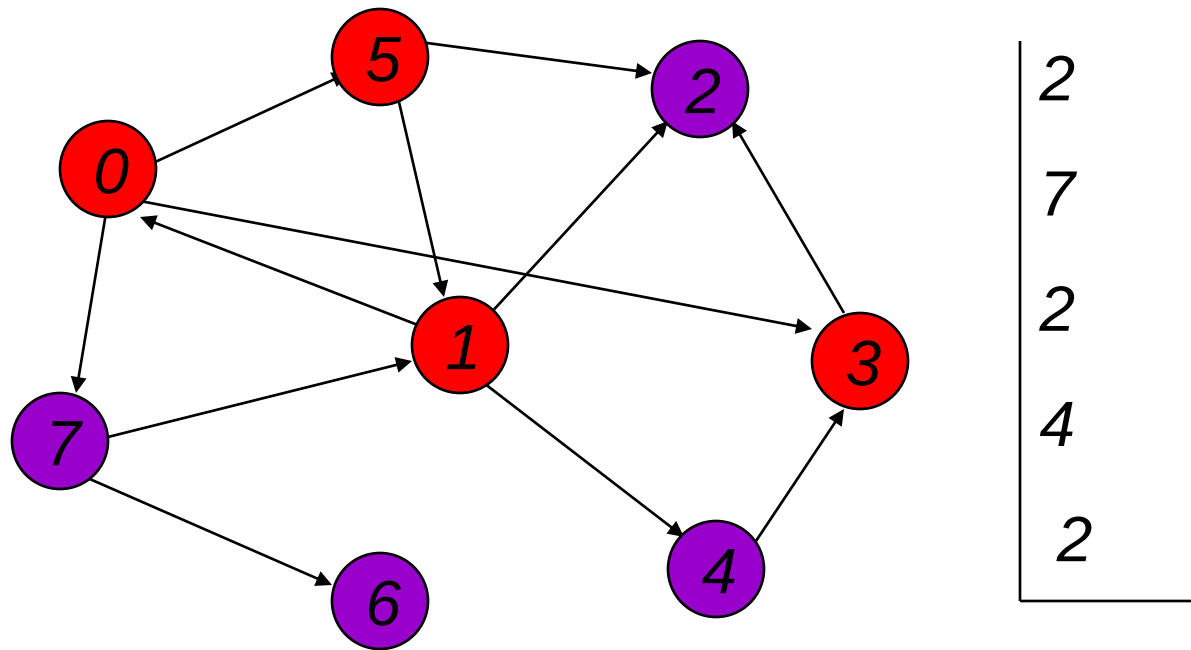


7
2
4
2

Pop/Visit/Mark 3

5 1 0 3

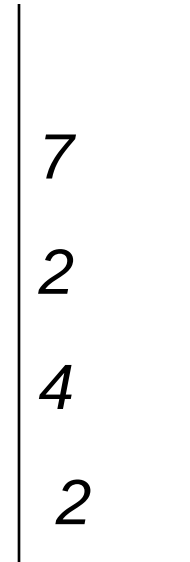
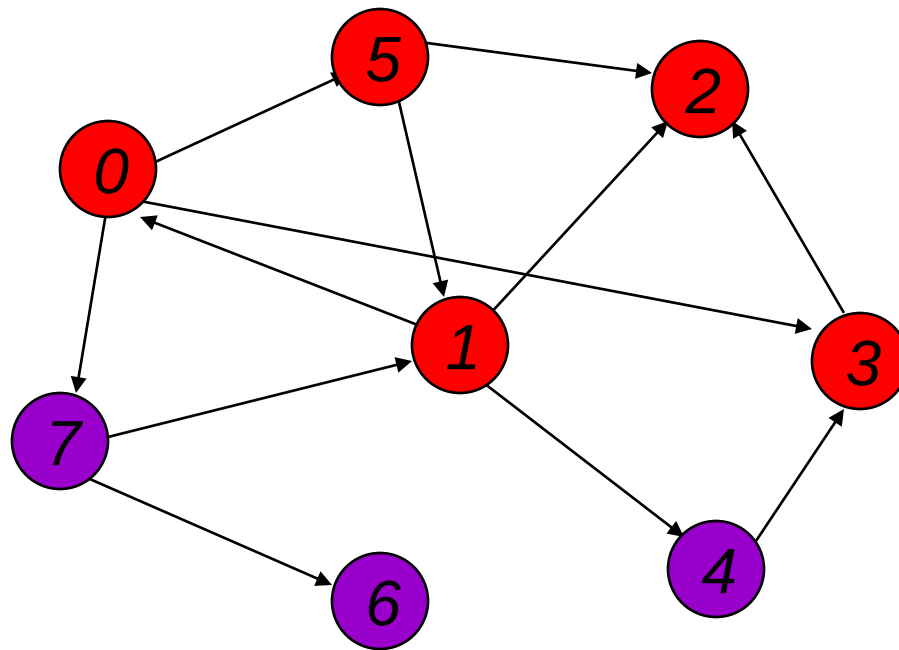
Preorder DFS: Start with Node 5



Push 2

5 1 0 3

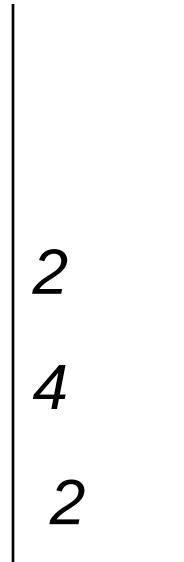
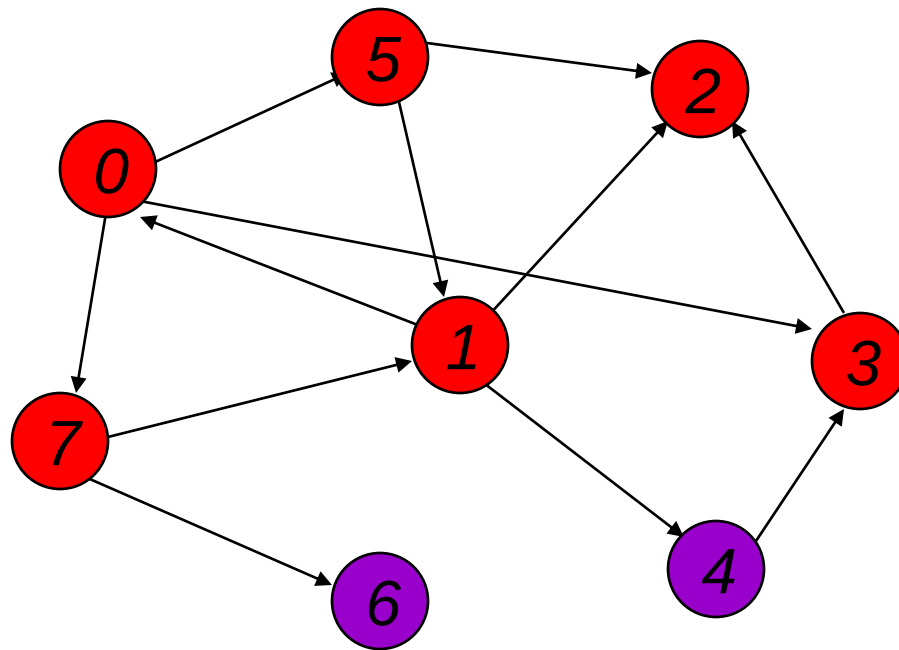
Preorder DFS: Start with Node 5



Pop/Mark/Visit 2

5 1 0 3 2

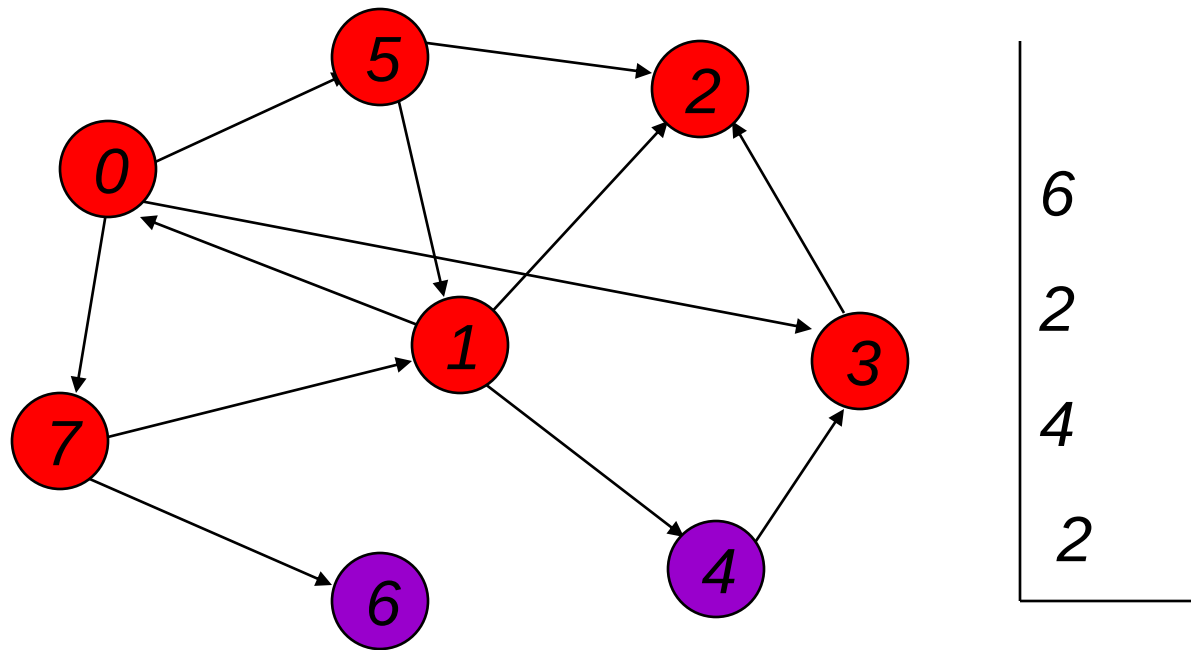
Preorder DFS: Start with Node 5



Pop/Mark/Visit 7

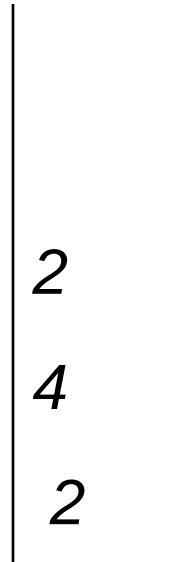
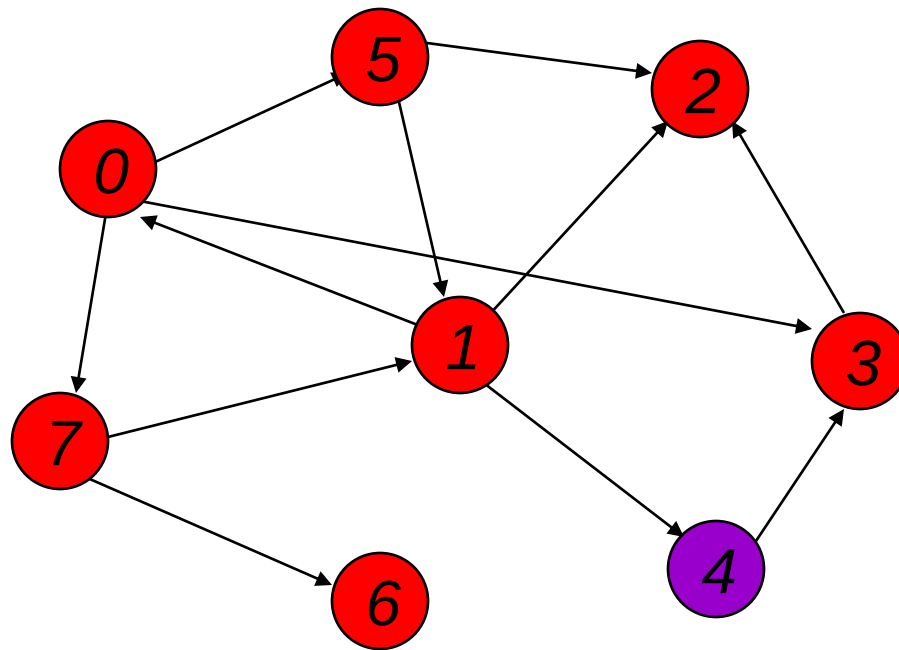
5 1 0 3 2 7

Preorder DFS: Start with Node 5



5 1 0 3 2 7

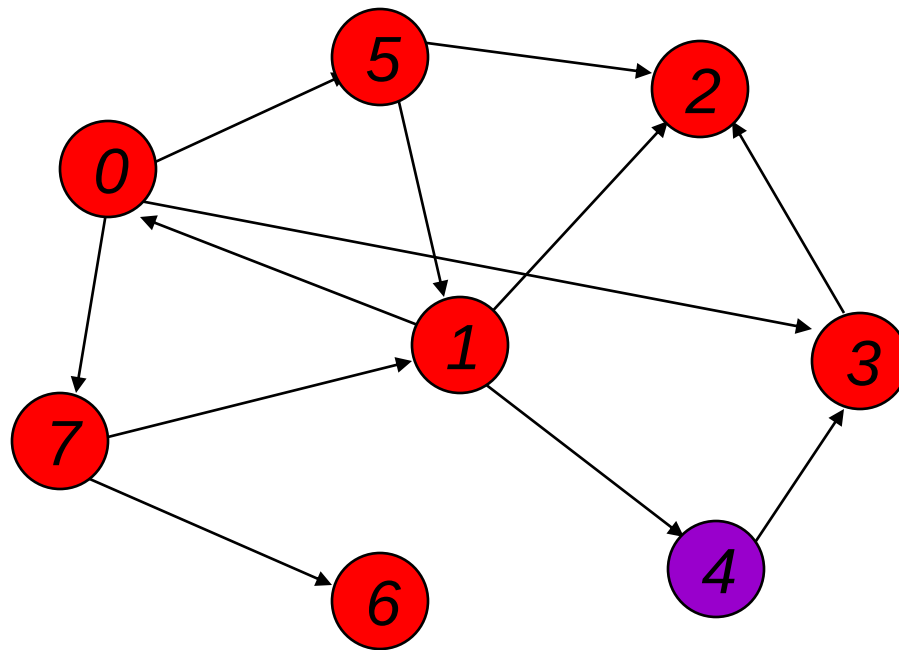
Preorder DFS: Start with Node 5



Pop/Mark/Visit 6

5 1 0 3 2 7 6

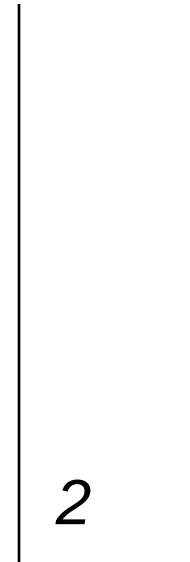
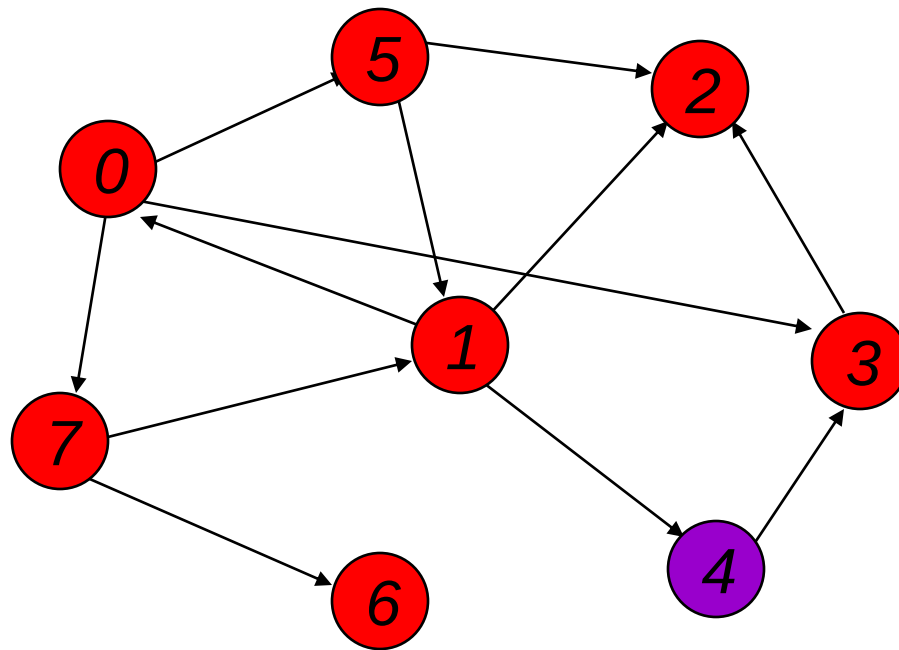
Preorder DFS: Start with Node 5



Pop (don't visit) 2

5 1 0 3 2 7 6

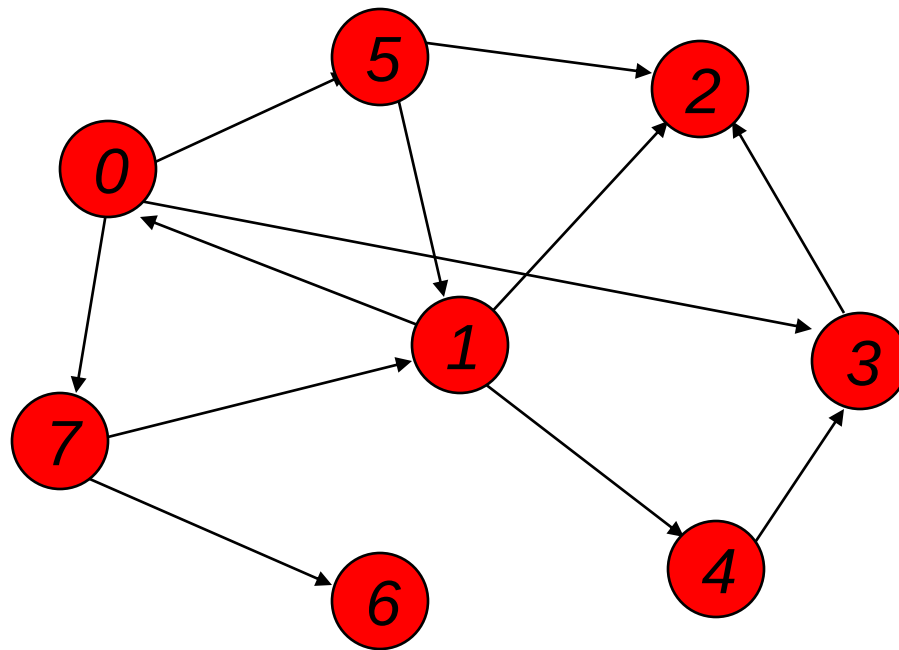
Preorder DFS: Start with Node 5



Pop/Mark/Visit 4

5 1 0 3 2 7 6
4

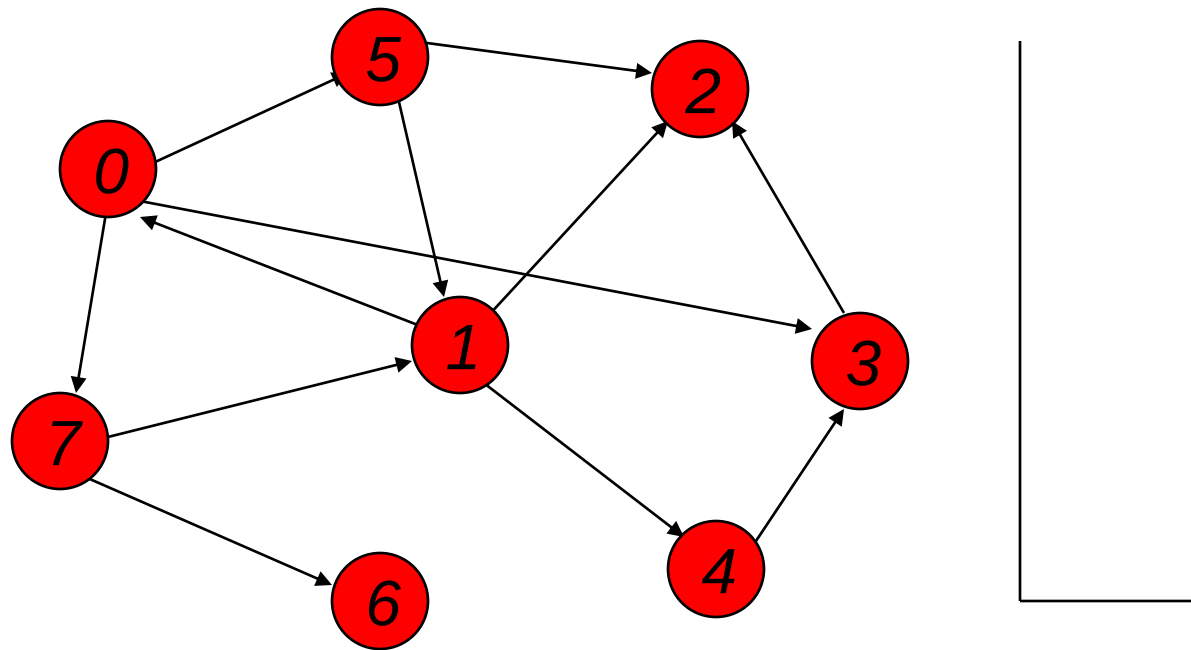
Preorder DFS: Start with Node 5



Pop (don't visit) 2

5 1 0 3 2 7 6
4

Preorder DFS: Start with Node 5

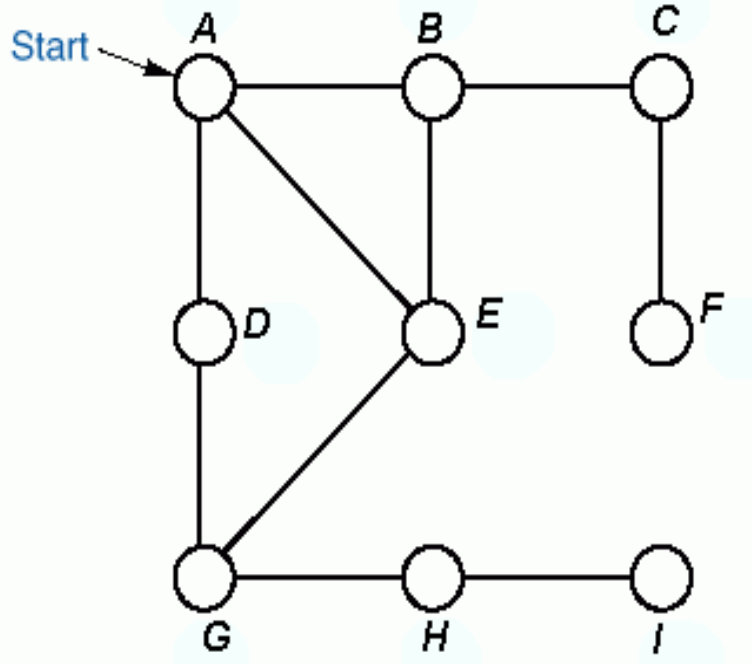


Done

5 1 0 3 2 7 6
4

Example

- Depth-first traversal using an explicit stack.

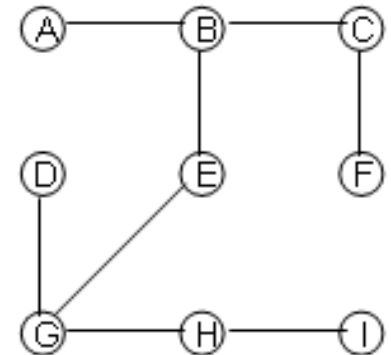


Stack

Order of
Traversal

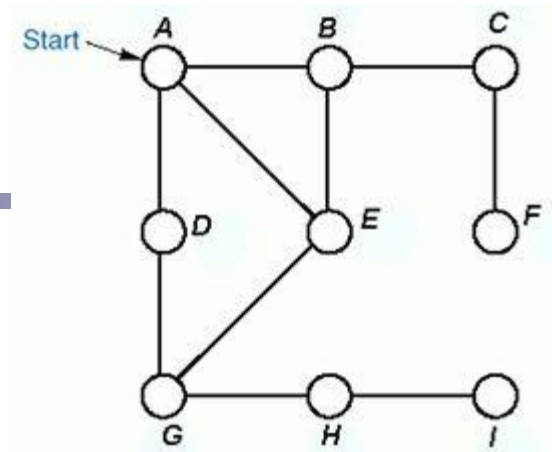
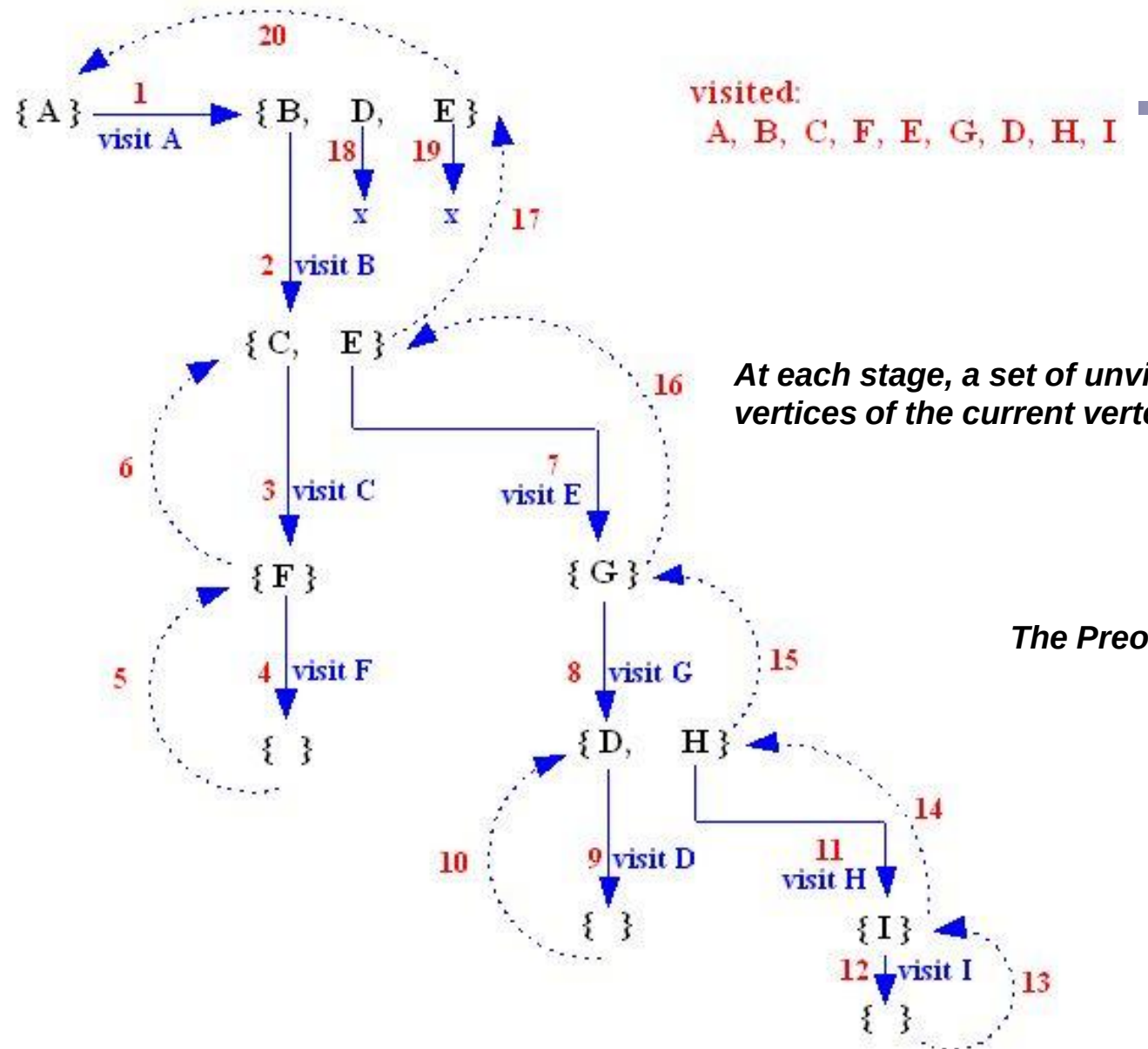
1	2	3	4	5	6	7	8	9
A	B	C	F	E	G	D	H	I

The Preorder Depth First Tree:



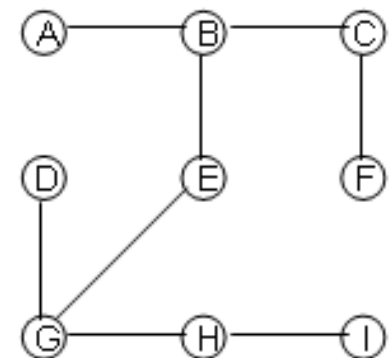
Note: The DFS-tree for undirected graph is a free tree

Recursive preorder Depth-First Traversal Tracing



At each stage, a set of unvisited adjacent vertices of the current vertex is generated.

The Preorder Depth First Tree:



Classification of the Edges of a Digraph

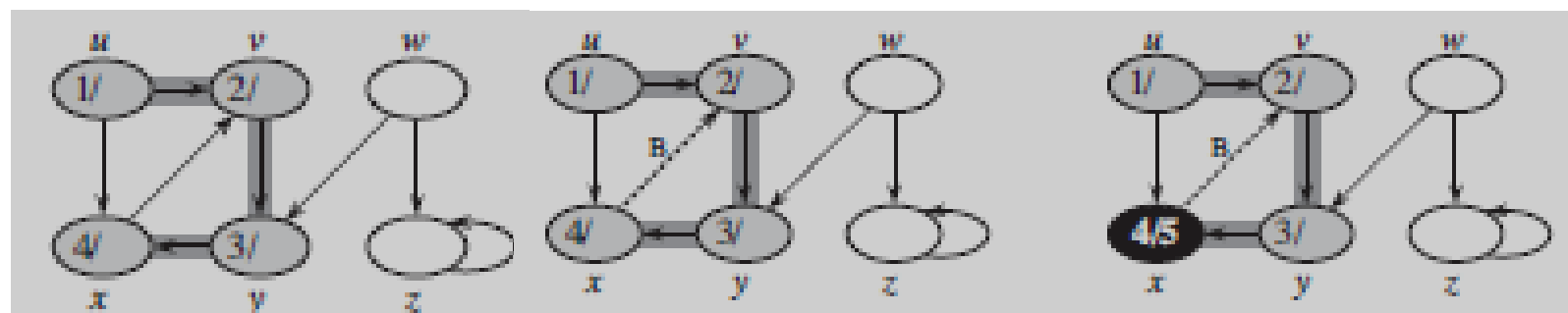
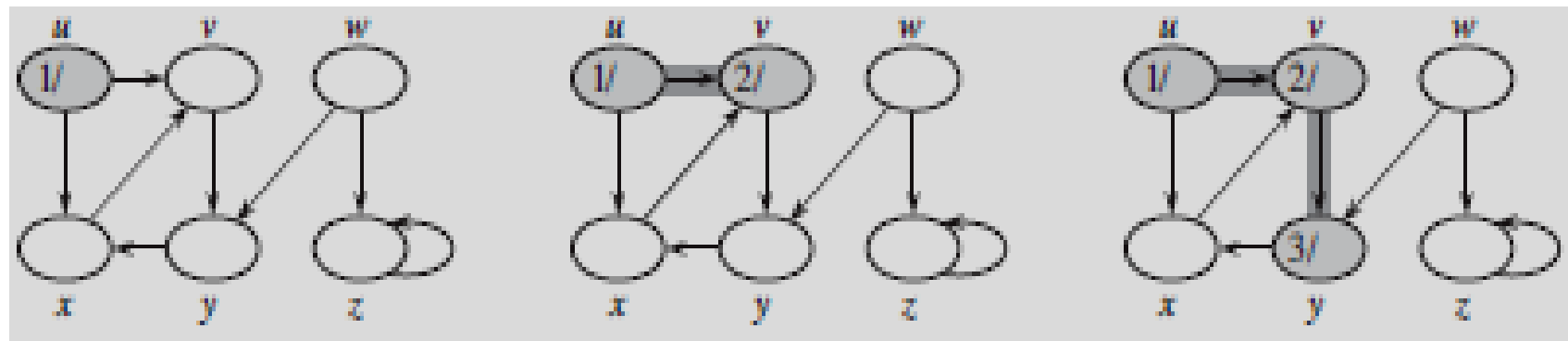
Tree edges: those on the DFS forest.

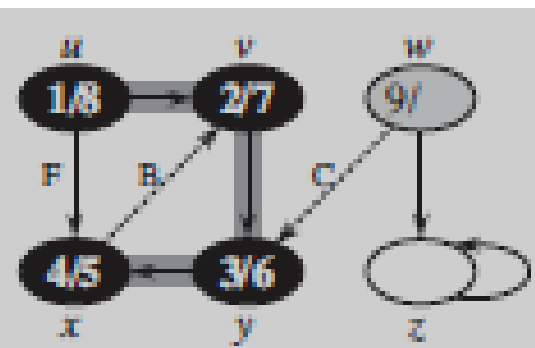
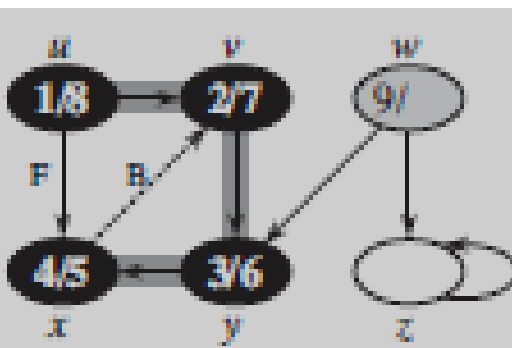
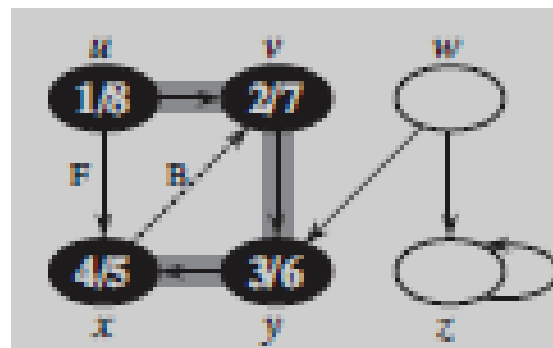
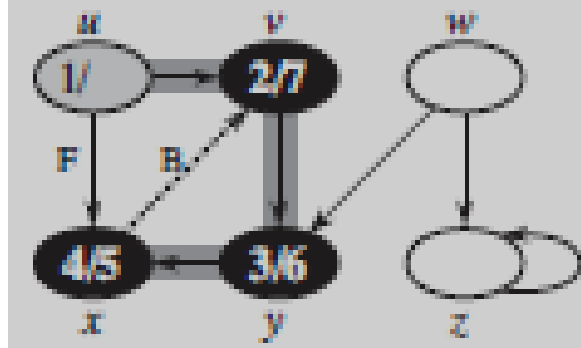
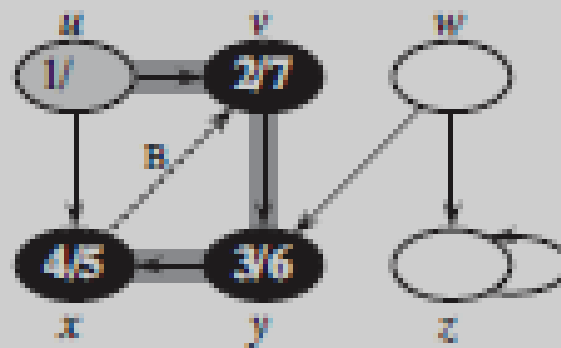
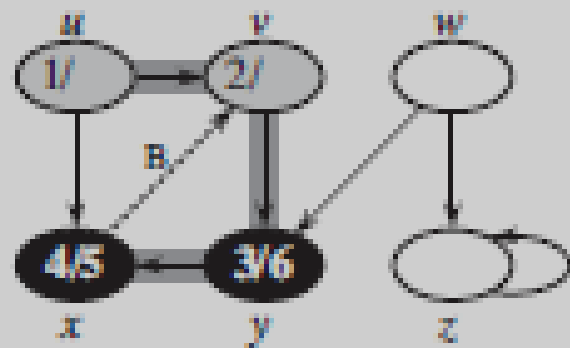
The **remaining** edges fall in three categories:

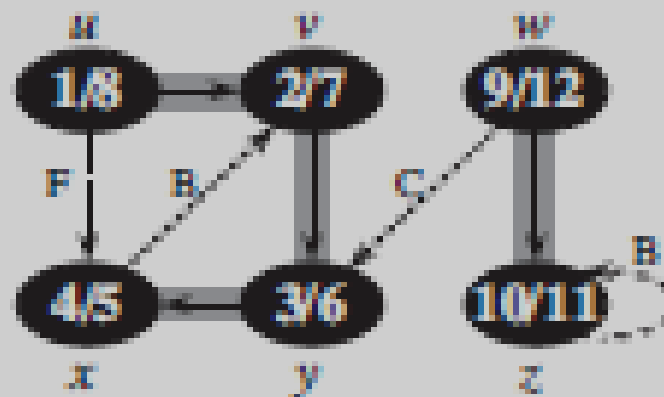
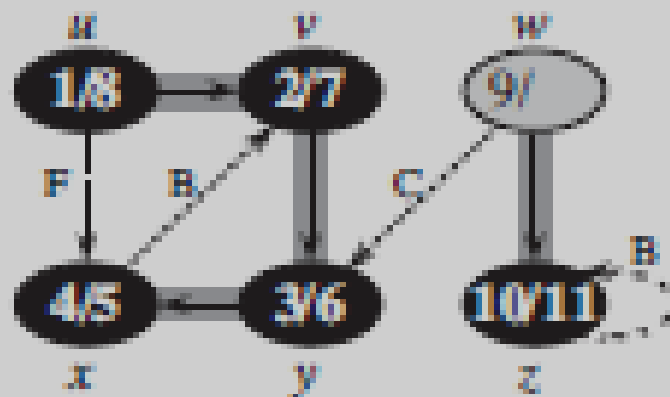
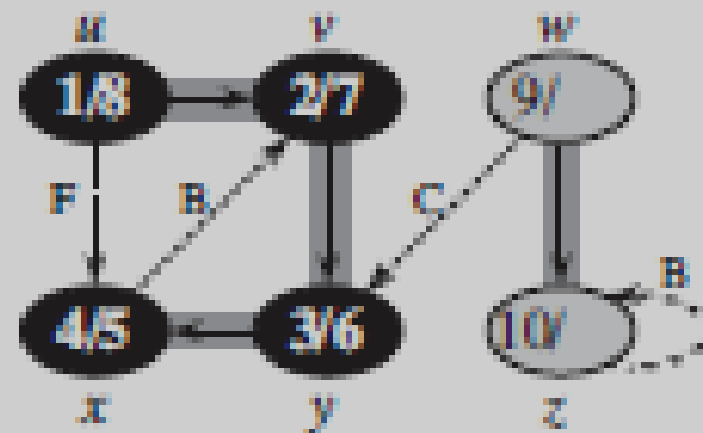
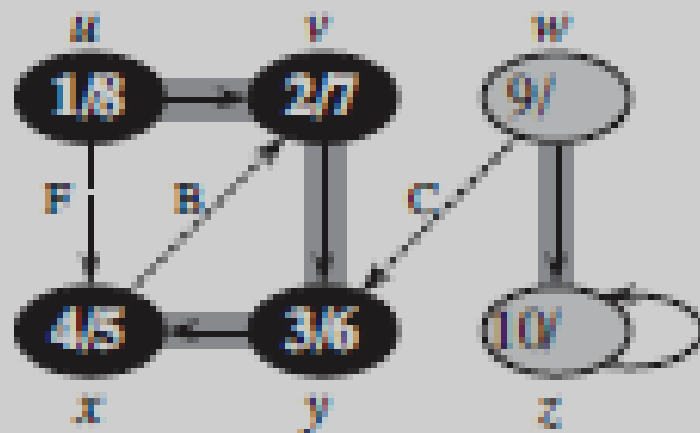
Forward edges: (u, v) where v is a **proper descendant** of u in the tree. [$u \neq v$.]

Back edges: (u, v) , where vertex v is an **ancestor** of u in the tree. [$u = v$ is allowed.]

Cross edges: (u, v) where u and v are **not ancestors** or **descendants** of one another (in fact, the edges may go between different trees).







Analysis of DFS

For a Graph $G=(V, E)$ and $n = |V|$ and $m=|E|$

- *When **Adjacency List** is used*
 - *Complexity is $O(m + n)$*
- *When **Adjacency Matrix** is used*
 - *Scanning each row for checking the connectivity of a Vertex is in order $O(n)$.*
 - *So, Complexity is $O(n^2)$*

DFS uses space $O(|V|)$ in the worst case to store the stack of vertices on the current search path as well as the set of already-visited vertices.